DESIGN LINEAR CONSTRAINED NEURAL LAYERS WITH IMPLICIT CONVEX OPTIMIZATION

Anonymous authors
Paper under double-blind review

ABSTRACT

One essential limitation of neural networks is how to enforce (hard) constraints on prediction. We propose a plug-in, differentiable layer, which involves a fast implicit (convex) optimization procedure to enforce the general linear constraint. It aims to minimize a divergence between unconstrained and constrained outputs. Connecting to and beyond existing handcrafted layers, we show that our layer degrades to classic layers like Softmax, Sinkhorn and tanh etc. when the corresponding constraint is enforced by KL-divergence minimization. We further show that by replacing the KL-div with a Euclidean distance, a closed-form solution can be derived for highly-efficient constraint enforcing. We evaluate the above two variants of layers, termed as BLCLayer and GLCLayer, with their corresponding neural solver BLCNet and GLCNet with simple MLP/GNN-like backbone. Experiments on liner programming, as well as two real-world problems: partial graph matching and portfolio allocation which involve other discrete constraints.

1 INTRODUCTION

Beyond deep neural networks for classification and regression, recent trends of deep learning have witnessed its expansion to diverse areas in which constraints are often required [6]. It remains an open problem [31] and calls for more general layers that can fulfill diversified constraint requirement.

One common approach to ensure that the network predictions satisfy certain constraints is through reinforcement learning (RL) [19]. RL has the capacity to handle very broad classes of constraints in an auto-regressive manner. However, it often faces issues such as large action spaces, sparse reward functions, and costive training and inference overhead. More importantly, it cannot always generalize well to new instances especially for serious scenarios e.g., chip design [11]. Another line of works e.g., DIFUSCO [30] resort to state-of-the-art training scheme e.g., diffusion models. The hope is that the constraints can be automatically satisfied during training. However, there is no theoretical guarantee and lacks an explicit scheme to control the output behavior. On the other hand, tailored neural nets are designed for special constraints like permutation [32] and partial permutation [31]. Still, the fact is that there exists no general scheme to handle arbitrary constraints.

On the other hand, linear constraints are powerful and widely used across areas, and a pertinent example is linear programming (LP) where both constraints and objective are linear w.r.t. the variables. Recently there are emerging efforts in dealing with certain linear constraints inside a neural network via a tailored layer. For example, a special yet important type of linear constraints, i.e., positive linear constraint is attained by altering the Sinkhorn algorithm with the resulting new layer in the seminal work LinSATNet [33]. As a preliminary technique, it yet suffers computational inefficiency and sometimes convergence instability. An improvement is made in GLinSAT [34], which introduces a differentiable projection layer via entropy-regularized LP. It advances the field by supporting general linear constraints, enabling GPU-accelerated batch processing without matrix factorization, and providing theoretical convergence guarantees via accelerated gradient descent. However, it incurs new practical challenges: Its accelerated gradient descent mechanism requires careful tuning of step-size hyperparameters that critically influence convergence speed and solution quality.

In this paper, we introduce LinConLayer (i.e., LCLayer for abbreviation), which designs the **Linear Constrained Layers** by minimizing the divergence between the layer's output and its input. We first provide a proof that existing classic layers, such as Softmax, Sigmoid, and Tanh, can be derived using implicit optimization regarding with minimizing specifically the KL-divergence. We then generalize

Table 1: Comparison of constraint enforcing layers. Note the GLinSAT involves gradient descent which is nontrivial especially when itself is an inner loop inside an outer loop of gradient descent i.e., network training. In contrast, our method is highly efficient and stable (e.g., see Table 5).

Methods	**output $\mathbf{x}$ range**	**constraint form**	**algorithm**	**efficiency**
LinSAT [33]	[0, 1]	positive linear	generalized Sinkhorn	low
GLinSAT [34]	[0, ∞]	linear	gradient descent	medium
BLCLayer (Sec. 2.2)	[0, ∞]	binary positive linear	Bregman projection	high
GLCLayer (Sec. 2.3)	[-∞, ∞]	linear	analytical solution	high

these special layers to the general case under the KL-divergence minimization framework and call the resulting layer as: Binary Linear Constrained Layers (BLCLayer), where the coefficients of the linear constraints are assumed binary. We adopt Bregman Iterative Projections algorithms [5], with sound convergence properties, to solve the implicit optimization problem. For general linear constraints, $\mathbf{W}\mathbf{x} = \mathbf{b}$, we replace the KL-divergence with Euclidean distance and derive a closed-form solution in each iteration without using gradient descent, to obtain the new general linear constraint layer (GLCLayer). Compared with [33], our GLCLayer no longer requires the coefficient matrices to be positive, and the final output entries can be arbitrary real numbers – see Table 1 for a comparison for the related methods. The pipeline is shown in Fig. 1 with the highlights as follows.

1) We propose designing constraint-satisfying networks using implicit optimization, in which fulfilling the output layer can be seen as solving a constrained convex optimization problem via differentiable matrix iteration. Our framework can incorporate existing layers like Softmax and Sinkhorn layers, as viewed as special cases w.r.t. certain constraints, when the objective involves KL divergence.

2) We extend Softmax and Sinkhorn layers by modifying their original constraints to binary positive linear constraints (BLC), leading to a new layer namely BLCLayer, whose implicit optimization can be efficiently solved by Bregman iterative projection without gradient descent. Experiments on the Partial Graph Matching and Portfolio Allocation tasks show that BLCNet (adding a backbone with our devised BLCLayer) performs much more stable and efficient than the tailored SOTA LinSATNet [33].

3) For the more general linear constraints (GLC), we devise the corresponding GLCLayer, achieved by replacing the KL divergence with Euclidean distance as the objective of implicit optimization. Based on GLCLayer, we design a neural solver with a neural backbone for linear programming, which shows high efficiency especially on large instances.

Notation. In line with Fig. 1, we define $\mathbf{y} \in \mathbb{R}^n$ as the unconstrained output of a network and $\mathbf{x} \in \mathbb{R}^n$ as its constrained one. It also holds in matrix form i.e., $\mathbf{Y}, \mathbf{X} \in \mathbb{R}^{n\times m}$ e.g., in graph matching.

Preliminaries. We discuss the background: Optimal Transport and Iterative Bregman Projections.

Optimal Transport. We first introduce Optimal Transport (OT) [21], which aims to find a mapping that minimizes the total cost of transporting mass from a source measure to a target measure. Kantorovich's formulation [17] extends the original problem by using probabilistic transport, which is now the standard approach. Given a cost matrix $\mathbf{C} \in \mathbb{R}_+^{m\times n}$ and histograms $(\mathbf{a}, \mathbf{b})$, where m and n represent the dimensions, the entropic regularized Kantorovich OT problem is formulated as:

$$\min_{\mathbf{X}\in U(\mathbf{a},\mathbf{b})} \langle \mathbf{C}, \mathbf{X}\rangle - \epsilon H(\mathbf{X}), \tag{1}$$

where $U(\mathbf{a}, \mathbf{b}) = \left\{\mathbf{X} \in \mathbb{R}_{mn}^+ \mid \mathbf{X}\mathbf{1}_n = \mathbf{a}, \mathbf{X}^\top \mathbf{1}_m = \mathbf{b}\right\}$ and $\epsilon > 0$ is the entropic regularization parameter. The entropy term $H(\mathbf{X}) = -\langle \mathbf{X}, \log \mathbf{X} - \mathbf{1}_{m\times n}\rangle$ enforces smoothness in the transport plan $\mathbf{X}$, which leads to a strongly convex objective, ensuring a unique solution that takes the form $\mathbf{X}_\epsilon^* = \text{diag}(\mathbf{u})\mathbf{K}\text{diag}(\mathbf{v})$, where $\mathbf{K} = e^{-\mathbf{C}/\epsilon}$ is the Gibbs kernel with scaling variables $(\mathbf{u}, \mathbf{v})$ [12].

Iterative Bregman Projections Bregman Iterative Projections provide an efficient way to solve convex optimization problems, including entropic optimal transport [5]. Specifically, given a positive matrix $\mathbf{K}$, it optimizes $\min_{\mathbf{X}\in\mathcal{C}} \text{KL}(\mathbf{X}\|\mathbf{K})$, where $\mathcal{C} = \bigcap_{l=1}^L \mathcal{C}_l$ is a convex set representing the constraints on $\mathbf{X}$. Here, we assume that $\mathcal{C}$ can be expressed as the intersection of L convex sets, and the minimization is performed by iterating over KL projections onto each constraint set $\mathcal{C}_l$. Starting from $\mathbf{X}^{(0)} = \mathbf{K}$, the iterations proceed as:

$$\mathbf{X}^{(l)} = \text{Proj}_{\mathcal{C}_n}^{\text{KL}}(\mathbf{X}^{(l-1)}) = \arg\min_{\mathbf{X}\in\mathcal{C}_l} \text{KL}(\mathbf{X}\|\mathbf{X}^{(l-1)}), \tag{2}$$

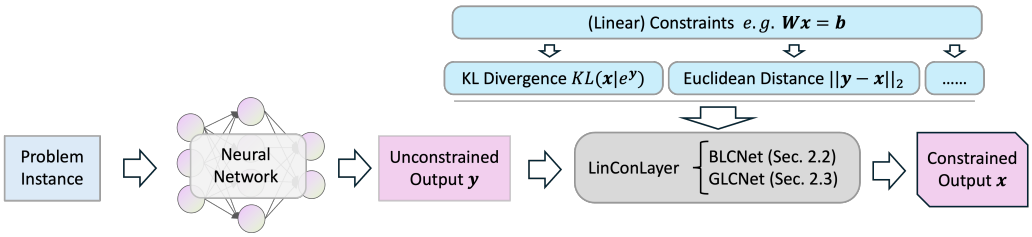


Figure 1: Pipeline: i) binary positive linear constraint version (BLC) and ii) general version (GLC).

As proven by Bregman [8], these projections converge to the optimal solution of the optimization problem as $n \to \infty$, yielding the solution:

$$\mathbf{X} = \text{Proj}_{\mathcal{C}}^{\text{KL}}(\mathbf{K}) = \lim_{n\to\infty} \text{Proj}_{\mathcal{C}_n}^{\text{KL}}(\mathbf{X}^{(n-1)}). \tag{3}$$

This method is commonly applied to solve entropic optimal transport problems and variants of OT when the cost matrix $\mathbf{C} \in \mathbb{R}_{nn}^{+}$ is used with the Gibbs kernel $\mathbf{K} = e^{-\mathbf{C}/\epsilon}$, and the constraints correspond to the marginal constraints in OT. The Sinkhorn algorithm [28], which is based on the iterative normalization of rows and columns of a matrix, is a specific application of the Bregman Iterative Projections. For example, in image matching and permutation problems, we set $\mathbf{X}^{(0)} = \mathbf{K}$ and $\mathcal{C} = \mathcal{C}_1 \bigcap \mathcal{C}_2$ where $\mathcal{C}_1 = \{\mathbf{X}|\mathbf{X}\mathbf{1} = \mathbf{1}\}$ and $\mathcal{C}_2 = \{\mathbf{X}|\mathbf{X}^\top\mathbf{1} = \mathbf{1}\}$, then according to Eq. 2 and Eq. 3, we can perform the Sinkhorn iterations as:

$$\mathbf{X}^{(2k+1)} = \mathbf{X}^{(2k)} \oslash \left(\mathbf{X}^{(2k)}\mathbf{1}_{n\times n}\right), \quad \mathbf{X}^{(2k+2)} = \mathbf{X}^{(2k+1)} \oslash \left(\mathbf{1}_{n\times n}\mathbf{X}^{(2k+1)}\right), \tag{4}$$

where $\oslash$ denotes element-wise division, and $\mathbf{1}_{n\times n}$ is the all-one matrix. This iterative procedure enforces doubly stochastic constraints and can be viewed as a Bregman projection applied to entropic optimal transport. It is commonly utilized as a layer in neural networks for tasks such as visual matching [32], ranking [20], and others [27].

2 LinConNet: Linear Constraint via Implicit Optimization Design

For clarity, we begin by reconsidering the classic layer from the perspective of KL divergence-based implicit optimization, and then modify the constraints to extend it to general cases. Specifically, given raw data $\mathbf{x}$ and a neural network $f_\theta(\cdot)$, one can obtain the unconstrained output vector $\mathbf{y}$. In this paper, our core task is to design a network such that the final output of the neural network satisfies (linear) constraints, such as $\mathbf{W}\mathbf{x} = \mathbf{b}$, where $(\mathbf{W}, \mathbf{b})$ are known.

2.1 Implicit Optimization View to Classic Constraint Layers: Softmax and Tanh

Inspired by the fact that the Sinkhorn Layer can be derived from constrained KL divergence, we propose that networks can be designed from the perspective of implicit optimization. First, we revisit several traditional layers (e.g., Softmax, Sigmoid, and Tanh) through the lens of implicit optimization and find that these networks can be viewed as special cases under constrained KL divergence.

Now we show that Softmax can also be understood from the implicit optimization view based on KL divergence. Specifically, given an unconstrained output $\mathbf{y}$, we consider the implicit optimization uder constraint $\mathcal{C} = \{\mathbf{x} \geq \mathbf{0}|\mathbf{x}^\top\mathbf{1} = 1\}$:

$$\min_{\mathbf{x}\geq\mathbf{0}} \text{KL}(\mathbf{x}|e^{\mathbf{y}/\epsilon}) \quad \text{s.t.} \quad \mathbf{x}^\top\mathbf{1} = 1, \tag{5}$$

Through the method of Lagrange multipliers, the above problem has a closed-form solution$\mathbf{x} = \frac{e^{\mathbf{y}/\epsilon}}{\sum_i e^{\mathbf{y}_i/\epsilon}}$, which is exactly the Softmax operator. Therefore, Softmax can be viewed as an activation layer derived from an implicit optimization involving KL divergence, with the constraint $\mathbf{x}^\top\mathbf{1} = 1$. Similarly, other layers, such as Sigmoid, can be reconsidered from the perspective of KL divergence. Unlike the constraints on matrix or vector summation seen in Sinkhorn or Softmax, the constraint for Sigmoid is restricted to $\mathbf{x} \in (0, 1)$. We construct this implicit optimization by defining the variable $1 - \mathbf{x}$, which can be formulated as:

$$\min_{\mathbf{x}} \text{KL}\left([\mathbf{x}, 1-\mathbf{x}] \parallel \left[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}\right]\right) \quad \text{s.t.} \quad \mathbf{x} \in (0, 1). \tag{6}$$

Table 2: "Implicit optimization perspective for Softmax, Sigmoid, Tanh layers, etc., given the unconstrained output $\mathbf{y}$ or $\mathbf{Y}$ of neural networks.

Layer	Formula	Optimization Objective	Constraints
Softmax	$\mathbf{x} = \frac{e^{\mathbf{y}/\epsilon}}{\sum_i e^{\mathbf{y}_i/\epsilon}}$	$\min_{\mathbf{x}\geq\mathbf{0}} \mathrm{KL}(\mathbf{x} \,\|\, e^{\mathbf{y}/\epsilon})$	$\mathbf{x}^\top\mathbf{1} = 1$
Sigmoid	$\mathbf{x} = \frac{1}{1+e^{-\mathbf{y}}}$	$\min_{\mathbf{x}} \mathrm{KL}\left([\mathbf{x}, 1-\mathbf{x}] \,\|\, \left[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}\right]\right)$	$\mathbf{x} \in (0,1)$
Tanh	$\mathbf{x} = \frac{e^{\mathbf{y}}-e^{-\mathbf{y}}}{e^{\mathbf{y}}+e^{-\mathbf{y}}}$	$\min_{\mathbf{x}} \mathrm{KL}\left(\left[\frac{\mathbf{x}+1}{2}, \frac{1-\mathbf{x}}{2}\right] \,\|\, [e^{\mathbf{y}}, e^{-\mathbf{y}}]\right)$	$\mathbf{x} \in (-1,1)$
Sinkhorn	See Eq. 4	$\min_{\mathbf{X}} \mathrm{KL}\left(\mathbf{X} \,\|\, \exp(\mathbf{Y}/\epsilon)\right)$	$\mathbf{X1} = \mathbf{1}, \mathbf{X}^\top\mathbf{1} = \mathbf{1}$
BLCLayer	See Eq. 9	$\min_{\mathbf{x}\geq\mathbf{0}} \mathrm{KL}(\mathbf{x} \,\|\, e^{\mathbf{y}/\epsilon})$	$\mathbf{Wx} = \mathbf{b}$
GLCLayer	See Eq. 12	$\min_{\mathbf{x}} \frac{1}{2}\|\|\mathbf{x}-\mathbf{y}\|\|^2$	$\mathbf{Wx} = \mathbf{b}$

The optimization problem seeks to align the probability vector $[\mathbf{x}, 1-\mathbf{x}]$ with the feature vector $[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}]$, ensuring that $\mathbf{x}$ remains within a valid probability range. This can also be solved using the method of Lagrange multipliers. In addition to the Sigmoid layer, the Tanh layer can also be understood from the perspective of KL divergence. A summary of this is presented in Table 2, with the proofs provided in Appendix B.3 and B.5.

2.2 BLCLayer: Binary Linear Constraint Layer via Bregman Projections

Based on KL-divergence, we now demonstrate how to generalize the Softmax and Sinkhorn layers to new layers that accommodate different constraints. In this paper, we focus on the linear constraint $\mathbf{Wx} = \mathbf{b}$ and propose the Binary Linear Constrained Layer (BLCLayer). Our key idea is that, since the Sinkhorn layer satisfies row and column sum constraints through Bregman iterations, it raises the question of whether it can be extended to handle more complex and general constraints. Initially, we consider the case where the constraint matrix $\mathbf{W} \in \{0,1\}^{m\times n}$ is binary, and aim to solve:

$$\min_{\mathbf{x}\geq\mathbf{0}} \mathrm{KL}(\mathbf{x} \mid e^{\mathbf{y}}) \quad \text{s.t.} \quad \mathbf{Wx} = \mathbf{b}. \tag{7}$$

Unlike Softmax, Sigmoid, and Tanh layers, having explicit solutions, the above implicit optimization has no closed-form solution. We need to adopt an approach similar to the Sinkhorn algorithm, using Bregman iterative projections to decompose the constraints and obtain its iterative solution. Specifically, we decompose the constraint set $\mathcal{C} = \{\mathbf{x} \geq 0 \mid \mathbf{Wx} = \mathbf{b}\}$ into m subsets of constraints: $\mathcal{C} = \bigcap_{l=1}^{L} \mathcal{C}_l$, where $\mathcal{C}_l = \{\mathbf{x} \geq 0 \mid \mathbf{w}_l^\top\mathbf{x} = \mathbf{b}_l\}$, and we define $\mathcal{C}_l = \mathcal{C}_{l+m}$ for $l < m$. By applying Bregman iterative projections, we solve it iteratively by optimizing the subproblems of Eq. 7:

$$\mathbf{x}^{(l+1)} = \arg\min_{\mathbf{x}\geq\mathbf{0}} \mathrm{KL}(\mathbf{x} \mid \mathbf{x}^{(l)}) \quad \text{s.t.} \quad \mathbf{w}_l^\top\mathbf{x} = \mathbf{b}_l \tag{8}$$

where $\mathbf{w}_l$ and $\mathbf{b}_l$ are the l-th components of the matrix $\mathbf{W}^\top$ and the vector $\mathbf{b}$, respectively, and we define $\mathbf{w}_{l+m} = \mathbf{w}_l$, $\mathbf{b}_{l+m} = \mathbf{b}_l$ for $l > 0$. By initializing $\mathbf{x}^{(0)} = e^{\mathbf{y}}$, and iterating l from 0 to a sufficiently large number, we can obtain Eq. 7 by solving Eq. 8. Then, to solve Eq. 8, since $\mathbf{w} \in \{0,1\}^n$, we can obtain the explicit solution to the optimization in Eq. 8 as follows:

$$\mathbf{x}^{(l+1)} = (\mathbf{1}_n - \mathbf{w}_l) \odot \mathbf{x}^{(l)} + \mathbf{w}_l \odot \frac{\mathbf{b}_l}{\mathbf{w}_l^\top\mathbf{x}^{(l)}}\mathbf{x}^{(l)} \tag{9}$$

We provide the derivation in Appendix B.6, and a convergence proof for the iteration described by Eq. 8 is provided in Appendix E. The above iteration, based on vector multiplication and division, corresponds to the BLCLayer for the linear constraint $\mathbf{Wx} = \mathbf{b}$.

Furthermore, consider the case of inequality constraints (e.g., $\mathbf{w}_l^\top\mathbf{x} \leq \mathbf{b}_l$). In [34], it converts the inequality into an equality constraint by introducing additional positive learnable parameters. Alternatively, the iteration can be adjusted as already done for partial Optimal Transport (OT). By replacing the constraint $\mathbf{w}_l^\top\mathbf{x}$ in Eq. 8 with the inequality $\mathbf{w}_l^\top\mathbf{x} \leq \mathbf{b}_l$, we can update the iteration:

$$\mathbf{x}^{(l+1)} = (\mathbf{1}_n - \mathbf{w}_l) \odot \mathbf{x}^{(l)} + \min\left\{1, \frac{\mathbf{b}_l}{\mathbf{w}_l^\top\mathbf{x}^{(l)}}\right\} \cdot \mathbf{w}_l \odot \mathbf{x}^{(l)}. \tag{10}$$

For the constraint $\mathbf{w}_l^\top\mathbf{x} = \mathbf{b}_l$, one only needs to replace the $\min$ operation with a $\max$ operation in Eq. 10." In practice, although the assumption of a matrix $\mathbf{W} \in \{0,1\}^{m\times n}$ may appear restrictive, it is sufficiently general to handle most graph-related scenarios. The experiments in Sec. 3.1 and

Sec. 3.2 provide two examples that demonstrate its capabilities. Of course, the BLCLayer does have some limitations, such as the requirement for positive outputs and the constraint coefficients being binary, in order to ensure explicit solutions for the subproblems. In next subsection, we discuss how to replace the KL divergence with alternative methods to accommodate more general constraints.

2.3 GLCLayer: General Linear Constraint Layer via Implicit Optimization

Recall that the above BLCLayer has limitations in that it handles only constraints where the variables are positive and its constraint parameters $\mathbf{W} \in \{0,1\}^{n\times m}$. To satisfy a more general satisfaction, we propose a general linear constraint layer designed for seamless integration into neural architectures.

Network Design for General Linear Satisfaction $\mathbf{Wx} = \mathbf{b}$. Given the equality constraints[1] $\mathbf{Wx} = \mathbf{b}$, we assume $m < n, rank(W) = m$ and there exist a feasible solution $\mathbf{x}_0$ that satisfying $\mathbf{Wx}_0 = \mathbf{b}$. Following the previous discussion, if we consider the KL divergence as the optimal implicit optimization objective function, there are two main issues: 1) The output $\mathbf{x}$ is necessarily non-negative, which may not be suitable for certain cases, such as regression problems, where negative values for $\mathbf{x}$ are allowed; and 2) Solving Eq. 7 or Eq. 8 inevitably introduces additional Lagrange multipliers, which are not conducive to backpropagation during the training process. A natural idea is to explore whether we can adopt alternative objective functions, such as the Euclidean distance, to derive a simpler layer that satisfies the constraints. Thus, we attempt to replace the KL divergence with the Euclidean distance for implicit opti mization as follows:

$$\min_{\mathbf{x}} \frac{1}{2}||\mathbf{x}-\mathbf{y}||^2 \quad \text{s.t.} \quad \mathbf{Wx} = \mathbf{b} \tag{11}$$

where $\mathbf{y}$ is the unconstrained output of neural network and $\mathbf{x}$ is the constrained output satisfying $\mathbf{Wx} = \mathbf{b}$. Thus, we can easily obtain the solution for the above implicit optimization as:

$$\mathbf{x} = \mathbf{y} - \mathbf{W}^\top(\mathbf{W}\mathbf{W}^\top)^{-1}(\mathbf{W}\mathbf{y} - \mathbf{b}) \tag{12}$$

The derivation details are provided in Appendix B.2. Note that $(\mathbf{W}, \mathbf{b})$ are known matrices or vectors, and therefore the training process does not involve computing gradients with respect to $(\mathbf{W}, \mathbf{b})$. Additionally, for this GLCLayer, we can compute the following:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}}\frac{\partial \mathbf{x}}{\partial \mathbf{y}} = (\mathbf{I} - \mathbf{W}^\top(\mathbf{W}\mathbf{W}^\top)^{-1}\mathbf{W})\frac{\partial \mathcal{L}}{\partial \mathbf{x}}. \tag{13}$$

Thus, ideally, the above solution can be obtained without iterations. Compared to other approaches for satisfying linear constraints $\mathbf{Wx} = \mathbf{b}$, our GLCLayer does not require the introduction of additional hyperparameters (e.g., Lagrange multipliers as often used in literature), which can affect gradient backpropagation and make the convergence of iterative algorithms difficult to control when implemented as a layer. The main limitation of this method, however, is that it requires computing the inverse of $\mathbf{W}\mathbf{W}^\top$ via an LU decomposition, which must be precomputed before the training and inference processes. It can also incur numerical instability.

Special Cases in $\mathbf{Wx} = \mathbf{b}, \mathbf{x} \geq 0$ for Linear Programming. For the linear constraint $\mathbf{Wx} = \mathbf{b}$, we naturally associate it with the linear programming problem: $\min_{\mathbf{x}} \mathbf{c}^\top\mathbf{x}$ s.t. $\mathbf{Wx} = \mathbf{b}$, $\mathbf{x} \geq 0$. In this context, we explore the possibility of optimizing this linear programming problem using a neural network that is linear and feasible. Clearly, while the constraints $\mathbf{Wx} = \mathbf{b}$, $\mathbf{x} \geq 0$ are linear, they can be reformulated as $\tilde{\mathbf{W}}\mathbf{x} = \mathbf{b}$. However, the matrix $\tilde{\mathbf{W}}$ corresponding to these modified constraints will always have fewer columns than rows, i.e., $n < m$, which implies that $\tilde{\mathbf{W}}\tilde{\mathbf{W}}^\top$ does not exist. In this case, we are forced to introduce additional variables to design the network. Specifically, for the LP problem, we modify the implicit optimization as follows:

$$\min_{\mathbf{x}} \mathbf{c}^\top\mathbf{x} + \frac{1}{2}||\mathbf{x}-\mathbf{y}||^2 \quad \text{s.t.} \quad \mathbf{Wx} = \mathbf{b},\ \mathbf{x} \geq 0 \tag{14}$$

We can solve the above optimization problem using the ADMM method. Specifically, by introducing the variable $\mathbf{z}$ with constraints $\mathbf{z} = \mathbf{x}$, the Lagrange multiplier λ, and the penalty parameter ρ, we can formulate the Lagrangian function:

$$L(\mathbf{x}, \mathbf{z}, \lambda) = \mathbf{c}^\top\mathbf{x} + \frac{1}{2}\|\mathbf{x}-\mathbf{y}\|^2 + \lambda^\top(\mathbf{Wx} - \mathbf{b}) + \rho\|\mathbf{x}-\mathbf{z}\|^2, \tag{15}$$

[1] Inequality constraints can be transformed to equality in the design of networks as discussed in [34].

then iteration can be updated as

$$\mathbf{x}^{(k+1)} = \frac{\mathbf{y} - \mathbf{c} + \mathbf{W}^\top \lambda^{(k)}}{1 + 2\rho\mathbf{y}},\ \mathbf{z}^{(k+1)} = \max\{0, \mathbf{x}^{(k+1)}\}, \lambda^{(k+1)} = \lambda^{(k)} + (\mathbf{W}\mathbf{x}^{(k+1)} - \mathbf{b}), \tag{16}$$

where k represents the iteration number. In the network design, we initialize $\mathbf{x}^{(0)} = \mathbf{y}$ and $\lambda^{(0)} = \mathbf{0}$, and then iteratively update based on Eq. 16 until convergence. From an optimization perspective, our layer design combines the parallel iterative algorithm of optimization with the prediction of the neural network. On the one hand, it simplifies the techniques traditional algorithm for convergence by learning a proximal point, and on the other hand, through supervised or unsupervised training, it leverages the neural network to accelerate inference rapidly.

3 Experiments

In line with [33] on positive linear constraint, we apply BLCLayer as a plug-in to the shared backbones across baseline methods, with two real-world tasks: partial graph matching and portfolio allocation in Sec. 3.1 and 3.2. In Sec. 3.3, we introduce a new task: learning LP with the proposed GLCLayer and further exploration. All experiments are implemented by PyTorch 2.5.1 on dual Intel(R) 8352V CPUs and an NV 4090 GPU (24GB). We integrate the layers into a backbone network and term the whole network solver as BLCNet and GLCNet (see Table 4).

Table 3: Mean F1, train and inference time of graph matching on Pascal VOC Keypoint (unfiltered setting).

GM Net	Constraint Technique	F1 ↑	Train ↓	Test ↓
BBGM	differentiable blackbox [24]	0.5516	1h 12m	1h 21m
NGMv2	Sinkhorn	0.5453	1h 25m	1h 34m
NGMv2	LinSAT-5 iters	0.5858	1h 56m	2h 15m
NGMv2	LinSAT-10 iters	0.5839	2h 19m	2h 37m
NGMv2	BLCLayer-5 iters (ours)	0.5972	1h 28m	1h 36m
NGMv2	BLCLayer-10 iters (ours)	0.6012	1h 37m	1h 46m

3.1 Experiments on Visual Graph Matching

Problem. Partial GM refers to the setting when the matching graphs contain different numbers of nodes. We follow [31] by formulating it as a top-k selection task with an estimated number of inliers and devised a top-k module that can be plugged into deep graph matching pipelines.

Constraint Formulation and Layer Design. For graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, where the cardinalities of the vertex sets are $|V_1| = n_1$ and $|V_2| = n_2$. $M \in \mathbb{R}^{n_1 \times n_2}$ is used to represent the potential correspondences between nodes in G_1 and G_2, with $M_{i,j}$ indicating the matching score between node i in G_1 and node j in G_2. In previous bijective graph matching networks, the one-to-one node matching constraint is enforced by the Sinkhorn algorithm. It formulates the matching problem by adding a partial matching constraint: assume that the number of matchable inliers is ϕ:

$$\mathbf{X}\mathbf{1}_{n_2} \le \mathbf{1}_{n_1}, \quad \mathbf{X}^\top \mathbf{1}_{n_1} \le \mathbf{1}_{n_2}, \quad \mathbf{1}_{n_1}\mathbf{X}\mathbf{1}_{n_2} = \phi, \quad \mathbf{X} \in \{0,1\}^{n_1 \times n_2} \tag{17}$$

Then with the procedure in Sec. 2.1, we get the iterations with the projections ($\mathbf{X}$ omitted):

$$\text{Proj}_{\mathcal{C}_1}^{KL} = \text{diag}(\min\left(\mathbf{1}_{n_1}, \frac{\mathbf{1}_{n_1}}{\mathbf{X}\mathbf{1}_{n_2}}\right))\mathbf{X}, \text{Proj}_{\mathcal{C}_2}^{KL} = \mathbf{X}\text{diag}(\min\left(\frac{\mathbf{1}_{n_2}}{\mathbf{X}^\top\mathbf{1}_{n_1}}, \mathbf{1}_{n_2}\right)), \text{Proj}_{\mathcal{C}_3}^{KL} = \frac{\phi\mathbf{X}}{\mathbf{1}_{n_1}^\top\mathbf{X}\mathbf{1}_{n_2}}. \tag{18}$$

Finally, as introduced by [5], assuming $\mathcal{C}_l = \mathcal{C}_{l+3}$ with positive integer l as the index of Bregman iteration, the minimization is solved with iteration: $\mathbf{X}^{(n)} = \text{Proj}_{\mathcal{C}_n}^{KL}\left(\mathbf{X}^{(n-1)}\right)$, starting from $\mathbf{X}^{(0)} = \mathbf{K} = e^{\mathbf{Y}/\epsilon}$. We can get the solution by $\mathbf{X}^* = \lim_{n\to\infty} \text{Proj}_{\mathcal{C}_n}^{KL}(\mathbf{X}^{(n-1)})$, which performs the iterations for Eq. 2 and [8] guarantees its convergence.

Table 4: Sharpe Ratio, Train/Inference Time (in seconds) for the SOTA Portfolio Allocation Methods [33] and different constraint techniques over iterations.

Layer	Sharpe ↑	Train ↓	Inference ↓
Unconstrained	2.190	**85.880**	8.990
LinSAT-10	2.508	976.080	73.000
LiSAT-100	2.755	4862.520	524.390
cvxpylayers	3.058	271.700	26.660
BLCNet-5	2.773	202.650	14.340
BLCNet-10	2.770	268.210	18.850
GLCNet	**3.053**	91.860	**7.200**

Experiment Details. We follow NGMv2 [32] as backbone, and replace the original Sinkhorn and LinSAT layer with BLCLayer. In NGMv2, a VGG16 [25] is used to extract node and global features, which are refined using SplineConv [13], while the edge features are derived from the node features and graph connections. Matching scores are further predicted using the neural graph matching network [32], resulting in a matrix denoting matching

prediction. This matrix is then inputted into our BLCLayer to enforce constraints from Eq. 17. Its output is reshaped into matrix for end-to-end training with permutation loss. During inference, the Hungarian algorithm is applied to the matching matrix to keep only the highest matching scores and discard the rest of the unpaired nodes.

The baselines: BBGM [26], Sinkhorn, LinSAT, and BLCNet share the same training configurations. We use a batch size 8 and train for 10 epochs, with each epoch of 1K iterations. Adam is used with a learning rate of 10^{-3} for the main network and a reduced rate of 10^{-5} for feature extractor, setting the temperature parameter τ to $\frac{1}{100}$. For reproducibility, we fix random seeds in all experiments. We also assess time cost for both training and inference.

Results. In Table 3, BLCNet outperforms BBGM, Sinkhorn-NGMv2, and all baselines intended for bijective matching and achieves results on par with LinSATNet, with considerably lower time. Our method's training and inference time are close to those of NGMv2 using Sinkhorn, yet with significantly better performance. Compared with LinSAT, it is slightly better while using far less compute. This gap amplifies when increasing iteration counts: LinSATNet (40 iters) gains only 0.0045% accuracy yet doubles overhead. We defer more details to Appendix C.1 for your reference.

Table 5: Total solving time, Bregman iteration time, and iteration cost ratio with varying problem scales. We set batch size to 8 and iteration number to 5 to align with previous settings in Table 3.

Max Scale	Total Time	Bregman Time	Bregman Ratio
10	0.1573	0.0119	7.57%
20	0.1786	0.0119	6.68%
40	0.2009	0.0119	5.93%
80	0.2244	0.0119	5.31%
Unlimited	0.2707	0.0117	4.32%

3.2 Experiments on Portfolio Allocation

Problem. It aims to select the optimal distribution of assets in financial markets, by designing a portfolio that balances returns against risks. Following LinSATNet, we use StemGNN [9] as the backbone with two branches, one predicting future asset prices and the other the portfolio. BLCLayer is applied to the portfolio prediction branch to enforce constraints. The network is supervised by a weighted sum of maximizing the Sharpe ratio and minimizing the prediction error of prices.

Constraint Formulation and Iterations in PA. Note that each asset is assigned a non-negative weight representing the proportion of the total investment allocated to that asset. The sum of these weights must equal 1, i.e., the sum-to-one constraint. Additionally, we introduce an expert preference constraint that requires the total allocation to a particular subset of assets, denoted by $\mathcal{C}$, to be at least a given threshold $p \in [0, 1]$. These constraints can be expressed as ($\mathbf{x}$ is the portfolio allocation): $\sum_{i=1}^{n} \mathbf{x}_i = 1$ and $\sum_{i \in \mathcal{C}} x_i \geq p$. The first constraint enforces the sum-to-one condition, while the second enforces the expert-driven preference for assets in set $\mathcal{C}$. Then we can build the BLCNet as discussed in Sec. 2.2 and GCLNet as in Sec. 2.3, with the same backbone among baselines.

Experimental Details. Given 120-day historical data, the network aims to build a portfolio with a maximized sharpe ratio for the next 120 days. The historical data includes real prices of 494 assets from the S&P 500 index from 2019-01-01 to 2020-12-30, while the model is tested on the real prices from 2021-03-01 to 2021-12-30. The subset of preferred stocks is set to C =(AAPL,MSFT, AMZN, TSLA, GOOGL, GOOG), and the preference ratio is set to p = 0.5.

Table 6: Mean values for small scale settings. 1600/800 training/test instances. The common backbone is three CNNs, encoding equality/inequality constraints and combining encoding results respectively (see Appendix D). Same for Table 7 and 8.

Problem scale			Method	Random Initialization			
var	eq	ineq		obj. score	obj. pct	time cost	time pct
500	400	0	Gurobi	-2919.75537	1	0.19156	1
			GLCNet	-2682.20407	0.91864	0.16213	0.84637
			GLCNet-UL	-2749.62122	0.94173	0.15202	0.79358
			GLCNet-SL	-2608.12988	0.89327	0.16748	0.87429
			GLCNet-SL-ADMM	**-2813.41788**	**0.96358**	0.13270	0.69271
500	0	400	Gurobi	-11803.58008	1	0.14538	1
			GLCNet	-10900.48817	0.92349	0.15910	1.09437
			GLCNet-UL	-11168.66551	0.94621	0.16519	1.13629
			GLCNet-SL	-10651.19656	0.90237	0.15387	1.05843
			GLCNet-SL-ADMM	**-11366.25744**	**0.96295**	0.17663	1.21495
500	200	200	Gurobi	-7506.81006	1	0.16777	1
			GLCNet	-6867.37998	0.91482	0.18245	1.08752
			GLCNet-UL	-7053.92421	0.93967	0.19188	1.14369
			GLCNet-SL	-6677.60782	0.88954	0.17837	1.06318
			GLCNet-SL-ADMM	**-7179.58821**	**0.95641**	0.20493	1.22147

For fairness, we strictly adhere to the configuration of LinSATNet, conducting 50 training epochs with a consistent batch size of 32 and a learning rate of 10^{-5} applied uniformly across all satisfiability layers. The training process employs a composite loss combining two components: 1) the mean squared error (MSE) for predicting future asset prices; and 2) the negative Sharpe ratio. To optimize performance while maintaining high inference speed, we implement constrained optimization with a

tolerance threshold of 10^{-3}, setting the temperature parameter τ to $\frac{1}{100}$ during training and reducing it to $\frac{1}{200}$for inference phases. For computational efficiency analysis, we separately measure the execution time for both training and inference stages, with training time encompassing full epoch iterations and inference time reflecting real-time prediction latency.

Results. In Table 4, BLCNet and GLCNet both achieve SOTA performance with high efficiency. Note that the relatively similar time cost for BLCNet and GLCNet is due to handling the inequality constraints would incur additional iterations for GLCNet despite it in theory has a closed-form solution. Further observations include: **1)** GLCNet-10 attains the highest Sharpe ratio (3.027). While LinSATNet exhibits acceptable performance with 100 iterations, it degrades severely when limited to 10 iterations (Sharpe ratio drops from 2.752 to 2.560). In contrast, BLCNet and GLCNet with 5 iterations produce slightly and significantly superior results compared to LinSATNet with 100 iterations. **2)** BLCNet reduces training time by 20× (224.48s vs 4450.98s) and inference time by 26× (17.20s vs 447.43s) over LinSAT-100, while achieving close performance. Similarly, GLCNet reduces training time by 17× (259.50s vs 4450.98s) and inference time by 30× (14.52s vs 447.43s) against LinSAT-100, while achieving better performance. **3)** The traditional unconstrained setting underperforms constrained methods by 0.457 Sharpe points. Refer to Appendix B.4 for more results.

3.3 Experiments on Linear Programming

Problem. Linear programming (LP) serves as a canonical framework for optimization:

$$\min_{\mathbf{x}\geq\mathbf{0}} \mathbf{c}^\top \mathbf{x} \text{ s.t. } \mathbf{W}\mathbf{x} = \mathbf{b} \qquad (19)$$

Traditional LP solvers employ sophisticated algorithms (e.g., simplex method, interior-point methods) that guarantee optimal solutions but suffer from exponential time complexity in worst cases. Recent works have explored neural networks to approximate LP solutions [1], yet an end-to-end net work is missing. In this part, GLCNet adopts a GNN backbone to serve as an end-to-end solver for LPs.

Table 7: Performance in **medium** scale settings for LP.

Problem scale			Method	Random Initialization			
var	eq	ineq		obj. score	obj. pct	time cost	time pct
1000	600	0	Gurobi	-14841.9209	1	0.65349	1
			GLCNet	-13421.69749	0.90431	0.21396	0.32741
			GLCNet-UL	-13821.09358	0.93122	0.20641	0.31586
			GLCNet-SL	-13190.90562	0.88876	0.22171	0.33927
			GLCNet-SL-ADMM	**-14156.81783**	**0.95384**	0.18600	0.28463
1000	0	600	Gurobi	-26753.94141	1	0.42943	1
			GLCNet	-24144.09443	0.90245	0.17717	0.41258
			GLCNet-UL	-25032.86036	0.93567	0.18955	0.44139
			GLCNet-SL	-23653.69468	0.88412	0.15770	0.36724
			GLCNet-SL-ADMM	**-25649.53871**	**0.95872**	0.18840	0.43871
1000	300	300	Gurobi	-20915.35547	1	0.50407	1
			GLCNet	-19002.85537	0.90856	0.22437	0.44512
			GLCNet-UL	-19580.53748	0.93618	0.22096	0.43835
			GLCNet-SL	-18643.32041	0.89137	0.21255	0.42167
			GLCNet-SL-ADMM	**-19862.05817**	**0.94964**	0.17970	0.45649

Protocols. Architecturally, GLCNet uses a GNN backbone over the variable–constraint bipartite graph. We compare with the commercial solver Gurobi, configured to run on 24 threads. We generate synthetic LP instances with varying scales (100–2,000 variables) and constraint compositions (equality/inequality ratios), ensuring feasibility via controlled generation: random feasible solutions are first sampled, constraints aligned with these solutions. For each instance, equality constraints are directly derived from solution values, while inequality constraints incorporate random offsets to maintain feasibility bounds. Invalid cases (e.g., unbounded problems) are filtered to ensure dataset validity. Afterwards, objective functions are randomly sampled from a normal distribution. To get the unconstrained output, we employ a dual-stream architecture with parallel 1D conv encoders for equality/inequality constraints. Each encoder processes constraint matrices via three Conv1D-BatchNorm-ReLU blocks, compressing input dimensions to 128 channels. Constraint features are fused via normalized weight-based matrix multiplication, combined with objective weights via channel concatenation, and projected to variable-aligned embeddings through a final Conv1D layer. It handles varying constraint counts (including no constraints) while maintaining dimensional consistency and explicit optimization awareness. The details are given in Appendix D.

Table 8: Performance in **large** scale settings for LP.

Problem scale			Method	Random Initialization			
var	eq	ineq		obj. score	obj. pct	time cost	time pct
2000	1600	0	Gurobi	-11699.65527	1	4.49247	1
			GLCNet	-10545.37271	0.90132	0.17327	0.03857
			GLCNet-UL	-10632.06912	0.90873	0.15715	0.03498
			GLCNet-SL	-10408.36663	0.88961	0.18522	0.04123
			GLCNet-SL-ADMM	**-10850.37729**	**0.92741**	0.16361	0.04974
2000	0	1600	Gurobi	-45422.77344	1	4.06018	1
			GLCNet	-40885.94683	0.90012	0.16675	0.04107
			GLCNet-UL	-40538.46261	0.89247	0.14954	0.03683
			GLCNet-SL	-41327.91041	0.90985	0.17577	0.04329
			GLCNet-SL-ADMM	**-42237.72857**	**0.92988**	0.15132	0.04288
2000	800	800	Gurobi	-28588.16211	1	4.04877	1
			GLCNet	-25995.50169	0.90931	0.16049	0.03964
			GLCNet-UL	-25831.11976	0.90356	0.14863	0.03671
			GLCNet-SL	-25384.57266	0.88794	0.17240	0.04258
			GLCNet-SL-ADMM	**-26437.47467**	**0.92477**	0.14527	0.04647

Four variants of the **GNN-backbone** GLCNet are compared: 1) **GLCNet** with random initialization on the backbone without any training – note the prediction would still satisfy the constraints via our projection layer; 2) **GLCNet-UL** trained via unsupervised learning where the goal is to maximize the objective score; 3) **GLCNet-SL** trained with supervised learning by minimizing a mean squared error (MSE) loss between its predictions and Gurobi-generated solutions as ground truth; 4) **GLCNet-SL-ADMM** started from the supervised model's prediction and apply a lightweight ADMM-based post-processing to refine the primal/dual variables. The training batch size is set to 128 and the Adam optimizer is used with a learning rate of 10^{-2} and cosine annealing scheduling over 3 epochs. There are four metrics: 1) objective score, measuring the raw optimization objective value; 2) objective percentage (pct), quantifying solution quality as the ratio of our attained score to Gurobi's reference value; 3) time cost, recording wall-clock execution time in seconds; and 4) time percentage (pct), comparing our runtime against Gurobi's one normalized to 1.

Table 9: Cross-scale generalization with test-time scale-up.

Problem scale			Method	Random Initialization			
var	eq	ineq		obj. score	obj. pct	time cost	time pct
1000	800	0	Gurobi	-5858.06	1	1.16462	1
			GLCNet	-4690.51	0.8108	0.16516	0.14181
			GLCNet-UL	-5076.31	0.8663	0.17056	0.14645
			GLCNet-SL	-5111.89	0.8712	0.16101	0.13825
			GLCNet-SL-ADMM	**-5601.01**	**0.9561**	0.17217	0.14783
1000	0	800	Gurobi	-23101.82	1	0.70701	1
			GLCNet	-19909.72	0.8850	0.17050	0.24116
			GLCNet-UL	-21568.04	0.9514	0.16133	0.22819
			GLCNet-SL	-21877.13	0.9565	0.16641	0.23537
			GLCNet-SL-ADMM	**-22274.77**	**0.9642**	0.11495	0.16259
1000	400	400	Gurobi	-13934.96	1	0.83830	1
			GLCNet	-11808.58	0.8705	0.18802	0.22429
			GLCNet-UL	-12739.52	0.9224	0.19274	0.22992
			GLCNet-SL	-12906.78	0.9349	0.19402	0.23144
			GLCNet-SL-ADMM	**-13261.90**	**0.9517**	0.09946	0.11864

Results. We evaluate three problem scales (small/medium/large) in Table 6-8. While GLCNet exhibits slower performance than Gurobi on small-scale problems, it shows significant efficiency advantages on medium- and large-scale instances. The solving time for Gurobi grows markedly with scale, whereas our method maintains stable efficiency, e.g., achieving solution targets with 0.95 success rate on large-scale problems while delivering $5\times$ to $20\times$ acceleration. Notably, the timing results are obtained with batch size 1. When deployed in parallel processing scenarios with GPU acceleration, it exhibits even greater performance over Gurobi's CPU-based implementation. This scalability makes GLCNet particularly suitable for modern computational environments requiring batch optimization and hardware acceleration.

Remark. Note our neural methods enjoy the capability of test-time handling a batch of LP instances simultaneously i.e., instance batch level parallelization, though for fairness we currently set the batch size by 1. This feature can further significantly reduce the mean solving time by enlarging the batch size, which is an off-the-shelf advantage for modern GPU clusters.

Table 10: Mixup of unsupervised and supervised learning.

Problem scale			Method	Random Initialization			
var	eq	ineq		obj. score	obj. pct	time cost	time pct
1000	800	0	Gurobi	-5858.06	1	1.16462	1
			GLCNet	-4690.51	0.8108	0.17279	0.14837
			GLCNet-UL	-5076.31	0.8663	0.17809	0.15292
			GLCNet-SL	-5111.89	0.8712	0.16964	0.14566
			GLCNet-SL-ADMM	**-5346.65**	**0.9127**	0.18572	0.15947
1000	0	800	Gurobi	-23101.82	1	0.70701	1
			GLCNet	-19956.04	0.8897	0.17663	0.24983
			GLCNet-UL	-21811.43	0.9441	0.16750	0.23692
			GLCNet-SL	-21647.91	0.9427	0.17260	0.24412
			GLCNet-SL-ADMM	**-21930.56**	**0.9493**	0.11512	0.16283
1000	400	400	Gurobi	-13934.96	1	0.83830	1
			GLCNet	-11854.92	0.8736	0.19463	0.23217
			GLCNet-UL	-12804.71	0.9189	0.19833	0.23658
			GLCNet-SL	-12861.47	0.9213	0.20109	0.23988
			GLCNet-SL-ADMM	**-12984.60**	**0.9318**	0.19036	0.20972

Further Studies. Building on the GNN backbone, we set two settings for LP: 1) generalization test across different problem scales, with test-time scale-up (TTS), i.e. enabling unsupervised training with backpropagation other than one-shot forward inference. 2) unsupervised pretraining with supervised post training. The results are in Table 9 and 10. It is noted that: 1) TTS effectively improves the performance at the cost of additional iterations; 2) unsupervised pretraining is useful especially considering the high cost of obtaining LP ground truth especially for large-scale instances.

4 Conclusion

We have presented a general framework for integrating linear constraints for neural networks, in a way of developing fast and differentiable layers. The experimental results on general LP with linear constraints, and special combinatorial optimization tasks showcase its cost-effectiveness.

ETHICS STATEMENT

This paper aims to advance the state of the art in linear constrained machine learning. While the research may entail various societal implications, we do not identify any that warrant specific emphasis in this paper.

REPRODUCIBILITY STATEMENT

All experimental results in the paper are reproducible, and the implementation code of ParetoRouter/-code for reproducing experimental results will be fully open sourced on Github after the paper is accepted.

LLM USAGE STATEMENT

The contribution of LLM in the work proposed in this article is limited to: 1. polishing given written statements; 2. reviewing the syntax for written sentences. We declare that no experimental data was generated/modified by LLM.

REFERENCES

[1] Brandon Amos and J. Zico Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Lerning*, 2017.

[2] Tu Anh-Nguyen, Joey Huchette, and Christian Tjandraatmadja. Learning generalized linear programming value functions. In *Conference on Neural Information Processing Systems*, 2024.

[3] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pp. 17981–17993, 2021.

[4] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations*, 2017.

[5] Jean-David Benamou, Guillaume Carlier, Marco Cuturi, Luca Nenna, and Gabriel Peyré. Iterative bregman projections for regularized transportation problems. *SIAM Journal on Scientific Computing*, 37(2):A1111–A1138, 2015.

[6] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[7] Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *Advances in Neural Information Processing Systems*, 2020.

[8] Lev M Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR computational mathematics and mathematical physics*, 7(3):200–217, 1967.

[9] Defu Cao, Yujing Wang, Juanyong Duan, Ce Zhang, Xia Zhu, Congrui Huang, Yunhai Tong, Bixiong Xu, Jing Bai, Jie Tong, and Qi Zhang. Spectral temporal graph neural network for multivariate time-series forecasting. In *Advances in Neural Information Processing Systems*, 2020.

[10] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[11] Ruoyu Cheng and Junchi Yan. On joint learning for solving placement and routing in chip design. In *Advances in Neural Information Processing Systems*, 2021.

[12] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. *arXiv preprint arXiv:1306.0895*, 2013.

[13] Matthias Fey, Jan Eric Lenssen, Frank Weichert, and Heinrich Müller. Splinecnn: Fast geometric deep learning with continuous b-spline kernels. In *IEEE conference on computer vision and pattern recognition*, 2018.

[14] Zhang-Hua Fu, Kai-Bin Qiu, and Hongyuan Zha. Generalize a small pre-trained model to arbitrarily large TSP instances. In *AAAI conference on artificial intelligence*, 2021.

[15] Simon Geisler, Johanna Sommer, Jan Schuchardt, Aleksandar Bojchevski, and Stephan Günnemann. Generalization of neural combinatorial solvers through the lens of adversarial robustness. In *International Conference on Learning Representations*, 2022.

[16] Chaitanya K. Joshi, Thomas Laurent, and Xavier Bresson. An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*, 2019.

[17] L Kantorovich. On the transfer of masses (in russian). 37(2):227–229, 1942.

[18] Elias Boutros Khalil, Bistra N. Dilkina, George L. Nemhauser, Shabbir Ahmed, and Yufen Shao. Learning to run heuristics in tree search. In *International Joint Conference on Artificial Intelligence*, 2017.

[19] Chang Liu, Zetian Jiang, Runzhong Wang, Junchi Yan, Lingxiao Huang, and Pinyan Lu. Revocable deep reinforcement learning with affinity regularization for outlier-robust graph matching. In *International Conference on Learning Representations*, 2024.

[20] Gonzalo Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with gumbel-sinkhorn networks, 2018.

[21] Gaspard Monge. Mémoire sur la théorie des déblais et des remblais. *Mem. Math. Phys. Acad. Royale Sci.*, pp. 666–704, 1781.

[22] Mohammadreza Nazari, Afshin Oroojlooy, Lawrence V. Snyder, and Martin Takáč. Reinforcement learning for solving the vehicle routing problem. In *Advances in Neural Information Processing Systems*, 2018.

[23] Chenhao Niu, Yang Song, Jiaming Song, Shengjia Zhao, Aditya Grover, and Stefano Ermon. Permutation invariant graph generation via score-based generative modeling. In *International Conference on Artificial Intelligence and Statistics*, 2020.

[24] Marin Vlastelica Pogančić, Anselm Paulus, Vit Musil, Georg Martius, and Michal Rolinek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations*, 2020.

[25] Hussam Qassim, Abhishek Verma, and David Feinzimer. Compressed residual-vgg16 cnn model for big data places image recognition. In *2018 IEEE 8th annual computing and communication workshop and conference (CCWC)*, pp. 169–175. IEEE, 2018.

[26] Michal Rolínek, Paul Swoboda, Dominik Zietlow, Anselm Paulus, Vít Musil, and Georg Martius. Deep graph matching via blackbox differentiation of combinatorial solvers. In *European Conference on Computer Vision*, 2020.

[27] Michael E. Sander, Pierre Ablin, Mathieu Blondel, and Gabriel Peyré. Sinkformers: Transformers with doubly stochastic attention, 2022.

[28] Richard Sinkhorn. A relationship between arbitrary positive matrices and doubly stochastic matrices. *The annals of mathematical statistics*, 35(2):876–879, 1964.

[29] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, 2015.

[30] Zhiqing Sun and Yiming Yang. Difusco: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, 2023.

[31] Runzhong Wang, Ziao Guo, Shaofei Jiang, Xiaokang Yang, and Junchi Yan. Deep learning of partial graph matching via differentiable top-k. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6272–6281, 2023.

[32] Runzhong Wang, Junchi Yan, and Xiaokang Yang. Neural graph matching network: Learning lawler's quadratic assignment problem with extension to hypergraph and multiple-graph matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.

[33] Runzhong Wang, Yunhao Zhang, Ziao Guo, Tianyi Chen, Xiaokang Yang, and Junchi Yan. LinSATNet: The positive linear satisfiability neural networks. In *International Conference on Machine Learning*, 2023.

[34] Hongtai Zeng, Chao Yang, Yanzhen Zhou, Cheng Yang, and Qinglai Guo. Glinsat: The general linear satisfiability neural network layer by accelerated gradient descent. In *Advances in Neural Information Processing Systems*, 2024.

APPENDIX

A RELATED WORKS

We first discuss works on handling constraints for neural networks, especially for constrained problems. We also introduce the involved techniques i.e., optimal transport and Bregman projection.

Differentiable Optimization Methods. These solvers aim to integrate traditional optimization methods with neural networks by utilizing backpropagation to compute gradients. Notably, [1] demonstrated the differentiability at optimal solutions by applying KKT conditions, particularly for quadratic programming. Concurrently, [2] proposed learning generalized linear programming value functions (GVFs) to approximate LP solutions, highlighting the role of implicit optimization in constraint satisfaction. Building on this foundation, further advancement [24; 7] focused on gradient approximation techniques. These methods allow combinatorial solvers to estimate gradients using forward passes or random perturbations, broadening the scope of differentiable optimization.

Heuristic Search. These methods focus on generating feasible solutions through search strategies. Two primary strategies have emerged: **Autoregressive search methods**, exemplified by models like [4] enforce hard constraints by sequentially constructing solutions, though their sequential nature introduces scalability bottlenecks. Meanwhile, **non-autoregressive search approaches** address scalability by assuming conditional independence among variables. Works like DIFUSCO [30] leverage heatmap-based search with iterative denoising for parallel decoding, but face inherent trade-offs between solution quality and multimodality [14; 16; 15].

Reinforcement Learning and Generative methods. Reinforcement learning (RL) represents one prominent line of research, especially applied to combinatorial optimization problems ranging from the traveling salesman problem [18], vehicle routing [22] to job scheduling [10]. While RL shows remarkable adaptability, its performance hinges on carefully engineered reward functions. A parallel research direction explores **diffusion models**, originally designed for continuous data generation [29], now adapted to discrete domains through binomial or multinomial perturbations [3]. These discrete diffusion frameworks have shown promise in constraint satisfaction tasks, particularly through works like [23], which developed permutation-invariant score-based models for graph-based optimization. Building on these foundations, recent efforts combine diffusion with graph neural networks [30], though computational complexity remains a barrier for NP-hard problems.

B CONSTRAINED LAYER DESIGN VIA DIVERGENCE BASED IMPLICIT OPTIMIZATION

B.1 DERIVATION OF BLCNET VIA KL DIVERGENCE-BASED IMPLICIT OPTIMIZATION

To find the probability distribution $\mathbf{x}$ that minimizes the Kullback-Leibler (KL) divergence subject to the constraint $\sum_i \mathbf{x}_i = 1$, we can use the method of Lagrange multipliers. The optimization problem is formally stated as:

$$\min_{\mathbf{x}} \mathrm{KL}(\mathbf{x}|\mathbf{y}) = \sum_{i=1}^{n} \left(\mathbf{x}_i \log \frac{\mathbf{x}_i}{k_i} - \mathbf{x}_i + k_i \right) \quad \text{s.t.} \sum_{i=1}^{n} \mathbf{x}_i = 1$$

Introduce the Lagrange multiplier λ and construct the Lagrangian function:

$$L(\mathbf{x}, \lambda) = \sum_{i=1}^{n} (\mathbf{x}_i \log \left(\frac{\mathbf{x}_i}{k_i} \right) - \mathbf{x}_i + k_i) + \lambda \left(\sum_{i=1}^{n} \mathbf{x}_i - 1 \right)$$

Take the partial derivative of $L(\mathbf{x}, \lambda)$ with respect to each $\mathbf{x}_i$:

$$\frac{\partial L}{\partial \mathbf{x}_i} = \log \left(\frac{\mathbf{x}_i}{k_i} \right) + \lambda = 0$$

Solving for $\mathbf{x}_i$, we get:

$$\log \left(\frac{\mathbf{x}_i}{k_i} \right) = -\lambda$$

Taking the exponential of both sides:

$$\frac{\mathbf{x}_i}{k_i} = e^{-\lambda} \implies \mathbf{x}_i = k_i e^{-\lambda}$$

Substitute $\mathbf{x}_i$ into the constraint $\sum_{i=1}^{n} \mathbf{x}_i = 1$:

$$\sum_{i=1}^{n} \mathbf{x}_i = \sum_{i=1}^{n} k_i e^{-\lambda} = e^{-\lambda} \sum_{i=1}^{n} k_i = 1$$

Solving for $e^{-\lambda}$, we get:

$$e^{-\lambda} = \frac{1}{\sum_{i=1}^{n} k_i}$$

Thus:

$$\mathbf{x}_i = k_i \cdot \frac{1}{\sum_{j=1}^{n} k_j}$$

The optimal solution is given by:

$$\mathbf{x}_i = \frac{k_i}{\sum_{j=1}^{n} k_j}$$

When k_i takes the exponential form $k_i = e^{\mathbf{y}_i/\tau}$ (where $\tau > 0$ is a constant), the optimal solution becomes:

$$\mathbf{x}_i = \frac{e^{\mathbf{y}_i/\tau}}{\sum_{j=1}^{n} e^{\mathbf{y}_j/\tau}},$$

which corresponds to the **softmax function** scaled by the parameter τ. Here, τ controls the sharpness of the distribution.

B.2 Derivation of GLCNet via Euclidean Distance-Based Implicit Optimization

To solve the constrained optimization problem

$$\min_{\mathbf{x}} \frac{1}{2}\|\mathbf{x} - \mathbf{y}\|^2 \quad \text{s.t.} \mathbf{W}\mathbf{x} = \mathbf{b}$$

, we employ the method of Lagrange multipliers. Construct the Lagrangian function with multiplier vector $\boldsymbol{\lambda}$:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = \frac{1}{2}\|\mathbf{x} - \mathbf{y}\|^2 + \boldsymbol{\lambda}^\top (\mathbf{W}\mathbf{x} - \mathbf{b})$$

Taking the gradient with respect to $\mathbf{x}$ and setting it to zero yields:

$$\nabla_{\mathbf{x}} \mathcal{L} = (\mathbf{x} - \mathbf{y}) + \mathbf{W}^\top \boldsymbol{\lambda} = 0$$

Solving for $\mathbf{x}$ gives the intermediate solution:

$$\mathbf{x} = \mathbf{y} - \mathbf{W}^\top \boldsymbol{\lambda}$$

Substituting this expression into the constraint $\mathbf{W}\mathbf{x} = \mathbf{b}$ produces:

$$\mathbf{W}\mathbf{y} - \mathbf{W}\mathbf{W}^\top \boldsymbol{\lambda} = \mathbf{b}$$

Rearranging terms to solve for the Lagrange multipliers, we obtain:

$$\mathbf{W}\mathbf{W}^\top \boldsymbol{\lambda} = \mathbf{W}\mathbf{y} - \mathbf{b}$$

Under the assumption that $\mathbf{W}\mathbf{W}^\top$ is invertible, the multipliers become:

$$\boldsymbol{\lambda} = (\mathbf{W}\mathbf{W}^\top)^{-1}(\mathbf{W}\mathbf{y} - \mathbf{b})$$

Substituting back into the expression for $\mathbf{x}$ yields the optimal solution:

$$\mathbf{x}^* = \mathbf{y} - \mathbf{W}^\top(\mathbf{W}\mathbf{W}^\top)^{-1}(\mathbf{W}\mathbf{y} - \mathbf{b})$$

To verify constraint satisfaction, we compute:

$$\mathbf{W}\mathbf{x}^* = \mathbf{W}\mathbf{y} - \mathbf{W}\mathbf{W}^\top(\mathbf{W}\mathbf{W}^\top)^{-1}(\mathbf{W}\mathbf{y} - \mathbf{b}) = \mathbf{b}$$

This confirms that the solution strictly satisfies the equality constraint $\mathbf{W}\mathbf{x} = \mathbf{b}$.

B.3 Derivation of Sigmoid function via KL Divergence Minimization

Consider the optimization problem:

$$\min_{\mathbf{x}} \mathrm{KL}\left([\mathbf{x}, 1-\mathbf{x}] \parallel \left[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}\right]\right) \quad \text{s.t.} \quad \mathbf{x} \in (0,1)$$

where the KL divergence is defined as:

$$\mathrm{KL}(\mathbf{X} \parallel \mathbf{Q}) = \sum_i \left(\mathbf{X}_i \log \frac{\mathbf{X}_i}{\mathbf{Q}_i} - \mathbf{X}_i + \mathbf{Q}_i\right).$$

Substituting $P = [\mathbf{x}, 1-\mathbf{x}]$ and $Q = \left[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}\right]$, the divergence expands to:

$$\mathrm{KL}(P \parallel Q) = \mathbf{x}\log\frac{\mathbf{x}}{e^{\mathbf{y}/2}} - \mathbf{x} + e^{\mathbf{y}/2} + (1-\mathbf{x})\log\frac{1-\mathbf{x}}{e^{-\mathbf{y}/2}} - (1-\mathbf{x}) + e^{-\mathbf{y}/2}.$$

Simplifying terms and removing constants independent of $\mathbf{x}$, the objective reduces to:

$$f(\mathbf{x}) = \mathbf{x}\log\mathbf{x} + (1-\mathbf{x})\log(1-\mathbf{x}) - \mathbf{y}\mathbf{x}.$$

To find the minimum, take the derivative of $f(\mathbf{x})$ with respect to $\mathbf{x}$:

$$\frac{df}{d\mathbf{x}} = \log\mathbf{x} - \log(1-\mathbf{x}) - \mathbf{y} = \log\frac{\mathbf{x}}{1-\mathbf{x}} - \mathbf{y}.$$

Setting the derivative to zero gives the critical condition:

$$\log\frac{\mathbf{x}}{1-\mathbf{x}} = \mathbf{y} \implies \frac{\mathbf{x}}{1-\mathbf{x}} = e^{\mathbf{y}}.$$

Solving for $\mathbf{x}$, we directly obtain the Sigmoid function:

$$\mathbf{x} = \frac{e^{\mathbf{y}}}{1+e^{\mathbf{y}}} = \frac{1}{1+e^{-\mathbf{y}}} = \sigma(\mathbf{y}).$$

B.4 Ablation Study for Portfolio Allocation

We also give the ablation test in Table 12.

This demonstrates that the Sigmoid function $\sigma(\mathbf{y})$ minimizes the KL divergence when $Q = \left[e^{\mathbf{y}/2}, e^{-\mathbf{y}/2}\right]$.

Table 11: F1, Train and Inference Time of Graph Matching on Pascal VOC Keypoint (Unfiltered).

GM Network	Constraint Technique	Max Iter	τ	F1	Train	Inference
BBGM	[24]	-	-	0.5516	1h 12m	1h 21m
NGMv2	Sinkhorn	10	0.01	0.5453	1h 25m	1h 34m
NGMv2	LinSAT	5	0.01	0.5858	1h 56m	2h 15m
NGMv2	LinSAT	10	0.01	0.5839	2h 19m	2h 37m
NGMv2	LinSAT	20	0.01	0.5830	3h 14m	3h 27m
NGMv2	LinSAT	40	0.01	0.5894	4h 52m	4h 58m
NGMv2	LinSAT	10	0.02	0.6007	2h 25m	2h 42m
NGMv2	LinSAT	10	0.1	0.5041	2h 21m	2h 37m
NGMv2	BLCNet	5	0.01	0.5972	1h 28m	1h 36m
NGMv2	BLCNet	10	0.01	0.6012	1h 37m	1h 47m
NGMv2	BLCNet	20	0.01	0.5937	1h 46m	1h 51m
NGMv2	BLCNet	10	0.02	0.5920	1h 36m	1h 41m
NGMv2	BLCNet	10	0.1	0.5921	1h 38m	1h 47m

Table 12: Sharpe Ratio, Train, and Inference Time (in Seconds) for the Portfolio Allocation Methods.

Layer	Train Max-Iter	Test Max-Iter	τ_{train}	τ_{test}	Sharpe Ratio	Train Time (sec)	Inference Time (sec)
Unconstrained	-	-	-	-	2.085	138.02s	7.77s
LinSAT	5	5	1/10	1/200	2.124	595.98s	44.45s
LinSAT	5	5	1/10	1/100	1.646	620.92s	45.80s
LinSAT	5	5	1/10	1/50	1.426	603.75s	43.81s
LinSAT	5	5	1/10	1/20	1.335	581.46s	42.99s
LinSAT	5	5	1/10	1/10	1.311	561.24	41.65
LinSAT	5	5	1/100	1/100	1.891	571.58s	40.62s
LinSAT	5	5	1/100	1/200	2.392	631.66s	47.69s
LinSAT	10	10	1/100	1/200	2.560	977.35s	69.02s
LinSAT	100	100	1/100	1/200	2.752	4450.98s	447.43s
BLCNet	1	1	1/100	1/200	2.808	170.37s	10.44s
BLCNet	5	5	1/10	1/200	2.660	229.58s	13.78s
BLCNet	5	5	1/10	1/100	2.381	229.87s	13.91s
BLCNet	5	5	1/10	1/50	2.249	229.58s	13.92s
BLCNet	5	5	1/10	1/20	2.191	233.60s	14.02s
BLCNet	5	5	1/10	1/10	2.175	198.40s	11.77s
BLCNet	5	5	1/100	1/200	2.758	224.48s	17.20s
BLCNet	1	10	1/100	1/200	2.780	150.70s	17.43s
BLCNet	10	1	1/100	1/200	2.773	274.07s	8.52s
BLCNet	10	10	1/100	1/200	2.755	303.57s	19.44s
GLCNet	5	5	-	-	3.000	259.60s	14.52s
GLCNet	10	10	-	-	3.027	403.67s	27.38s

B.5 Derivation of $\tanh(\mathbf{y})$ via KL Divergence Minimization

To solve the optimization problem of minimizing the KL divergence given $\mathbf{y}$, we start by defining the probability distributions:

$$\mathbf{X} = \left[\frac{\mathbf{x}+1}{2}, \frac{1-\mathbf{x}}{2}\right], \quad \mathbf{Q} = \left[e^{\mathbf{y}}, e^{-\mathbf{y}}\right]$$

The optimization target is defined as:

$$\min_{\mathbf{x}} \mathrm{KL}(\mathbf{X} \parallel \mathbf{Q}) \qquad \mathbf{X}_i \geq 0$$

The KL divergence is defined as:

$$\mathrm{KL}(\mathbf{X} \parallel \mathbf{Q}) = \sum_{i=1}^{2} \left[\mathbf{X}_i \log \frac{\mathbf{X}_i}{\mathbf{Q}_i} - \mathbf{X}_i + \mathbf{Q}_i\right]$$

Substituting the components of P and Q, we expand the objective function:

$$\begin{aligned}\mathrm{KL}(P \parallel Q) = &\frac{\mathbf{x}+1}{2} \log\left(\frac{\mathbf{x}+1}{2}\right) - \frac{\mathbf{x}+1}{2} \log e^{\mathbf{y}} - \frac{\mathbf{x}+1}{2} + e^{\mathbf{y}} \\ &+ \frac{1-\mathbf{x}}{2} \log\left(\frac{1-\mathbf{x}}{2}\right) - \frac{1-\mathbf{x}}{2} \log e^{-\mathbf{y}} - \frac{1-\mathbf{x}}{2} + e^{-\mathbf{y}}\end{aligned}$$

Simplifying using $\log e^{\mathbf{y}} = \mathbf{y}$ and combining terms:

$$\begin{aligned}\mathrm{KL}(P \parallel Q) &= \frac{\mathbf{x}+1}{2}\log\left(\frac{\mathbf{x}+1}{2}\right) + \frac{1-\mathbf{x}}{2}\log\left(\frac{1-\mathbf{x}}{2}\right) - \frac{(\mathbf{x}+1)\mathbf{y}}{2} + \frac{(1-\mathbf{x})\mathbf{y}}{2} - 1 + e^{\mathbf{y}} + e^{-\mathbf{y}} \\ &= \left[\frac{\mathbf{x}+1}{2}\log\left(\frac{\mathbf{x}+1}{2}\right) + \frac{1-\mathbf{x}}{2}\log\left(\frac{1-\mathbf{x}}{2}\right)\right] - \mathbf{x}\mathbf{y} - 1 + e^{\mathbf{y}} + e^{-\mathbf{y}}\end{aligned}$$

Taking the derivative with respect to $\mathbf{x}$ and setting it to zero:

$$\frac{d}{d\mathbf{x}}\mathrm{KL} = \frac{1}{2}\log\left(\frac{\mathbf{x}+1}{1-\mathbf{x}}\right) - \mathbf{y} = 0$$

Solving this equation:

$$\begin{aligned}\log\left(\frac{\mathbf{x}+1}{1-\mathbf{x}}\right) &= 2\mathbf{y} \\ \frac{\mathbf{x}+1}{1-\mathbf{x}} &= e^{2\mathbf{y}} \\ \mathbf{x} &= \frac{e^{2\mathbf{y}}-1}{e^{2\mathbf{y}}+1} = \tanh(\mathbf{y})\end{aligned}$$

Therefore, the optimal solution is:

$$\mathbf{x} = \tanh(\mathbf{y})$$

B.6 Proof of Eq. 9

Now, given that $\mathbf{w}_l \in \{0,1\}^n$, we aim to solve the following optimization problem:

$$\min_{\mathbf{x}\geq\mathbf{0}} \mathrm{KL}(\mathbf{x} \mid \mathbf{x}^{(l)}) \quad \text{s.t.} \quad \mathbf{w}_l^\top \mathbf{x} = \mathbf{b}_l$$

We introduce a Lagrange multiplier λ_l for the constraint $\mathbf{w}_l^\top \mathbf{x} = \mathbf{b}_l$. The Lagrangian for this problem is:

$$\mathcal{L}(\mathbf{x}, \lambda_l) = \sum_{i=1}^{n}\left(x_i \log\frac{x_i}{x_i^{(l)}} - x_i + x_i^{(l)}\right) + \lambda_l(\mathbf{w}_l^\top \mathbf{x} - \mathbf{b}_l)$$

The gradient of the Lagrangian with respect to $\mathbf{x}$ is:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \log\frac{x_i}{x_i^{(l)}} + \lambda_l w_{li} = 0$$

This simplifies to:

$$x_i = x_i^{(l)} e^{-\lambda_l w_{li}}$$

- If $w_{li} = 0$, we have:

$$x_i = x_i^{(l)} = (1 - w_{li}) \cdot x_i^{(l)}$$

This means that when $w_{li} = 0$, the component x_i is not updated and remains equal to its initial value $x_i^{(l)}$.

- If $w_{li} = 1$, we get:

$$x_i = x_i^{(l)} e^{-\lambda_l}$$

This means that when $w_{li} = 1$, the component x_i is updated according to the exponential factor $e^{-\lambda_l}$. Since we know that $\mathbf{w}_l^\top \mathbf{x} = \mathbf{b}_l$, we can write:

$$\mathbf{w}_l^\top \mathbf{x} = \sum_i x_i = e^{-\lambda_l} \left(\sum_i x_i^{(l)} \right) = \mathbf{b}_l$$

Thus:

$$e^{-\lambda_l} = \frac{\mathbf{b}_l}{\sum_i x_i^{(l)}} = \frac{\mathbf{b}_l}{\mathbf{w}_l^\top \mathbf{x}^{(l)}}$$

Therefore, we have:

$$x_i = x_i^{(l)} \cdot \frac{\mathbf{b}_l}{\mathbf{w}_l^\top \mathbf{x}^{(l)}} = w_{li} x_i^{(l)} \cdot \frac{\mathbf{b}_l}{\mathbf{w}_l^\top \mathbf{x}^{(l)}}$$

Combining the cases for $w_{li} = 0$ and $w_{li} = 1$, we arrive at the final solution:

$$\mathbf{x}^{(l+1)} = (\mathbf{1}_n - \mathbf{w}_l) \odot \mathbf{x}^{(l)} + \mathbf{w}_l \odot \frac{\mathbf{b}_l}{\mathbf{w}_l^\top \mathbf{x}^{(l)}} \mathbf{x}^{(l)}$$

In this expression, $\mathbf{w}_l$ is a binary vector that determines which components of $\mathbf{x}$ are updated. For components where $w_{li} = 0$, the corresponding x_i remains unchanged, while for components where $w_{li} = 1$, the update is computed as $x_i = x_i^{(l)} \cdot \frac{\mathbf{b}_l}{\mathbf{w}_l^\top \mathbf{x}^{(l)}}$.

C Ablation Studies

C.1 Ablation Study for Graph Matching

We give the ablation results in Table 11.

C.2 Additional LP Comparison with HardNet

GLCLayer is closely related to projection-based architectures such as HardNet: in our LP setting, both approaches enforce affine equality constraints and non-negativity by projecting an unconstrained output onto the feasible set. To quantify their relative performance, we implemented a HardNet-based LP model and compared it against GLCNet under identical backbone, data, and training protocol.

In our implementation, GLCLayer is realized as a specialized PyTorch module that exploits the fact that the LP constraint matrix W is fixed: we precompute a Cholesky factorization of $WW^\top$ once and perform all subsequent solves as batched dense linear algebra on the GPU, with essentially no per-instance setup cost. For the HardNet-based model, we follow a generic usage pattern of a differentiable QP layer and invoke a generic solver interface to realize the projection. This offers greater generality, but incurs additional problem-construction and solver overhead (often on the CPU) and provides fewer opportunities for aggressive batching.

Table 13 summarizes the results on our LP benchmark for three problem sizes with different combinations of equality and inequality constraints. We report the objective value achieved by each method and the corresponding ratio to the optimal value obtained by a commercial LP solver.

Across all sizes and constraint mixes, GLCNet achieves higher objective ratios than the HardNet-based model. At the same time, due to the specialized implementation of GLCLayer, GLCNet trains substantially faster: on our hardware, one epoch for the HardNet-based model takes on the order of 10 minutes even on small LP instances, whereas one epoch of GLCNet requires only about 30–40 seconds. We emphasize that this speed gap reflects our respective implementations rather than a fundamental difference in asymptotic complexityboth layers implement Euclidean projections onto affine constraint sets of comparable dimension.

Table 13: Comparison between GLCNet (with GLCLayer) and a HardNet-based model on the LP benchmark. "Vars" denotes the number of decision variables, "Eq" and "Ineq" the numbers of equality and inequality constraints, respectively. "Obj." is the achieved objective value, and "Obj.%" is the ratio to the optimal value returned by a ground-truth LP solver (higher is better).

				GLCNet		HardNet-based	
Size	Vars	Eq	Ineq	Obj.	Obj.%	Obj.	Obj.%
Small	500	400	0	-2813.42	0.9636	-2690.41	0.9215
Small	500	0	400	-11366.26	0.9630	-10808.18	0.9157
Small	500	200	200	-7179.59	0.9564	-6817.91	0.9082
Medium	1000	600	0	-14156.82	0.9538	-13321.22	0.8975
Medium	1000	0	600	-25649.54	0.9587	-24409.23	0.9124
Medium	1000	300	300	-19862.06	0.9496	-18721.75	0.8951
Large	2000	1600	0	-10850.38	0.9274	-10289.26	0.8795
Large	2000	0	1600	-42237.73	0.9299	-40138.74	0.8837
Large	2000	800	800	-26437.47	0.9248	-24829.96	0.8685

Table 14: Profiling the cost of matrix inversion on the partial graph matching and portfolio allocation tasks. "Proj." denotes the total projection time per forward pass, and "Inv." the time spent in the matrix inversion / linear solve.

Task	Avg. proj. time (ms)	Avg. inv. time (ms)	Inv. ratio (%)
Graph matching	148.5520	0.0003	0.0002
Portfolio	2.6887	0.0001	0.0001

We also verified that both pipelines, when combined with the same ADMM refinement procedure, satisfy nearly all linear constraints up to a fixed numerical tolerance comparable to that of the ground-truth solver on all test instances. Thus, the improved training efficiency of GLCNet does not come at the expense of infeasible solutions.

C.3 Profiling the Cost of Matrix Inversion in BLCLayer

One potential concern for BLCLayer is the cost of computing matrix inverses involving $WW^\top$. Howerver, in our applications, the rank and dimension of W are very small, and the inversion step is negligible in practice compared to the rest of the network computation.

As an illustration, Table 14 reports the profiling results on the partial graph matching and portfolio allocation tasks. For each forward pass, we measure (i) the total time spent in the BLCLayer projection and (ii) the time spent specifically on the $WW^\top$ inversion / linear solve, and average these over many batches.

In both cases the inversion accounts for a vanishingly small fraction of the overall projection time. For our LP experiments, the constraint matrices are larger but still structured; we exploit this by precomputing a factorization of $WW^\top$ once and reusing it across all training and inference steps. Compared to classical LP solvers that repeatedly solve large linear systems (e.g., in simplex or interior-point iterations), GLCLayer only needs this preprocessing step once, and can amortize the cost over many problem instances and forward passes.

When scaling to larger problems, two primary factors influence resource consumption:

- **Constraint Sparsity.** Real-world constraint matrices W are often highly sparse. Leveraging sparse tensor formats and graph-based encoders significantly reduces storage and computation. Empirically, we find that problem dimensions up to $\mathcal{O}(10^4)$ remain feasible within a single training pipeline.

- **High-dimensional Outputs.** For extremely high-dimensional output vectors x, memory bottlenecks can be mitigated via block-wise autoregressive strategies. By partitioning x and enforcing constraints locally, the model can scale to massive industrial tasks.

Consequently, our layer benefits directly from sparse linear algebra, multi-GPU training, and sequential prediction architectures.

D BACKBONE FOR GLCNET

The backbone for GLCNet is shown in 1.

Listing 1: **Backbone for GLCNet**

```
class BipartiteGraphConvolution(nn.Module):
    """
    Partial bipartite graph convolution (either left-to-right or right-to-left)
    .
    """
    def __init__(self, emb_size, activation, initializer,
                 right_to_left=False, gather_first=False):
        super().__init__()

        self.emb_size = emb_size
        self.activation = activation
        self.right_to_left = right_to_left
        self.gather_first = gather_first

        # feature layers
        self.feature_module = nn.Sequential(
            nn.Linear(in_features=emb_size, out_features=self.emb_size),
            self.activation,
            nn.Linear(in_features=self.emb_size, out_features=self.emb_size),
        )
        # output_layers
        self.output_module = nn.Sequential(
            nn.Linear(in_features=emb_size * 2, out_features=self.emb_size),
            self.activation,
            nn.Linear(in_features=self.emb_size, out_features=self.emb_size),
        )

        # Apply initializer
        for module in [self.feature_module, self.output_module]:
            for layer in module:
                if isinstance(layer, nn.Linear):
                    initializer(layer.weight)
                    if layer.bias is not None:
                        nn.init.zeros_(layer.bias)

    def forward(self, inputs):
        left_features, edge_indices, edge_features, right_features = inputs

        if self.right_to_left:
            scatter_dim, gather_dim = 0, 1
            prev_features, gather_feautures = left_features, right_features
        else:
            scatter_dim, gather_dim = 1, 0
            prev_features, gather_feautures = right_features, left_features

        # compute joint features
```

```
        if self.gather_first:
            gathered = torch.index_select(gather_feautures, dim=0, index=
                edge_indices[gather_dim])
            joint_features = self.feature_module(torch.cat([edge_features,
                gathered], dim=1))
        else:
            gather_features = self.feature_module(gather_feautures)
            joint_features = edge_features * torch.index_select(
                gather_features, dim=0, index=edge_indices[gather_dim]
            )

        # perform convolution
        conv_output = torch.zeros(
            prev_features.shape[0], self.emb_size, device=prev_features.device
        )
        conv_output.index_add_(0, edge_indices[scatter_dim], joint_features)

        # output layer
        output = self.output_module(torch.cat([conv_output, prev_features], dim
            =1))

        return output


class ExtendedBipartiteGraphConvolution(nn.Module):
    def __init__(self, emb_size, activation, initializer, right_to_left=False,
        gather_first=False):
        super().__init__()

        self.emb_size = emb_size
        self.activation = activation
        self.right_to_left = right_to_left
        self.gather_first = gather_first

        # feature layers
        self.feature_module = nn.Sequential(
            nn.Linear(in_features=emb_size, out_features=self.emb_size),
            self.activation,
            nn.Linear(in_features=self.emb_size, out_features=self.emb_size),
        )
        self.feature_module_self = nn.Sequential(
            nn.Linear(in_features=emb_size, out_features=self.emb_size, bias=
                False),
            self.activation,
            nn.Linear(in_features=self.emb_size, out_features=self.emb_size),
        )

        # output_layers
        self.output_module = nn.Sequential(
            nn.Linear(in_features=emb_size * 2 + emb_size, out_features=self.
                emb_size),
            self.activation,
            nn.Linear(in_features=self.emb_size, out_features=self.emb_size),
        )

        # Apply initializer
        for module in [self.feature_module, self.feature_module_self, self.
            output_module]:
            for layer in module:
```

```
            if isinstance(layer, nn.Linear):
                initializer(layer.weight)
                if layer.bias is not None:
                    nn.init.zeros_(layer.bias)

    def forward(self, inputs):
        left_features, edge_indices, edge_features, right_features, self_edges
            = inputs

        if self.right_to_left:
            scatter_dim, gather_dim = 0, 1
            prev_features, gather_feautures = left_features, right_features
        else:
            scatter_dim, gather_dim = 1, 0
            prev_features, gather_feautures = right_features, left_features

        # compute joint features
        if self.gather_first:
            gathered = torch.index_select(gather_feautures, dim=0, index=
                edge_indices[gather_dim])
            joint_features = self.feature_module(torch.cat([edge_features,
                gathered], dim=1))
        else:
            gather_features = self.feature_module(gather_feautures)
            joint_features = edge_features * torch.index_select(
                gather_features, dim=0, index=edge_indices[gather_dim]
            )

        # perform convolution
        conv_output = torch.zeros(
            prev_features.shape[0], self.emb_size, device=prev_features.device
        )
        conv_output.index_add_(0, edge_indices[scatter_dim], joint_features)

        # self convolution
        self_out = None
        if isinstance(self_edges, torch.Tensor): # Q_matrix
            if self.gather_first:
                raise ValueError("Q in matrix is not supported for mixed-
                    integer extension.")
            self_out = self.feature_module_self(prev_features)
            self_out = torch.matmul(self_edges, self_out)
        elif isinstance(self_edges, tuple) and
            len(self_edges) == 2: # self_inds, self_feats
            self_inds, self_feats = self_edges
            if self.gather_first:
                self_gathered = torch.index_select(prev_features, dim=0, index=
                    self_inds[1])
                self_gathered = self.feature_module_self(torch.cat([self_feats,
                     self_gathered], dim=1))
            else:
                self_gathered = self_feats * torch.index_select(
                    self.feature_module_self(prev_features),
                    dim=0,
                    index=self_inds[1]
                )
            self_out = torch.zeros(
                prev_features.shape[0], self_gathered.shape[1], device=
                    prev_features.device
```

```
            )
            self_out.index_add_(0, self_inds[0], self_gathered)

        # output layer
        to_concat = [prev_features, conv_output]
        if self_edges is not None:
            to_concat += [self_out]
        output = self.output_module(torch.cat(to_concat, dim=1))

        return output
```

E CONVERGENCE PROOF

Proposition 1. *The sequence $\{\mathbf{x}^{(k)}\}$ generated by the Bregman iterative projection method converges to the unique solution $\mathbf{x}^*$ of Eq. 7 provided that the feasible set $\mathcal{C}$ is nonempty.*

Proof. The KL-divergence satisfies a fundamental identity known as the three-point property:

$$\mathrm{KL}(\mathbf{x} \mid \mathbf{z}) = \mathrm{KL}(\mathbf{x} \mid \mathbf{y}) + \mathrm{KL}(\mathbf{y} \mid \mathbf{z}) + \langle \nabla f(\mathbf{y}) - \nabla f(\mathbf{z}), \mathbf{x} - \mathbf{y} \rangle, \tag{20}$$

where $f(\mathbf{x}) = \sum_i (x_i \log x_i - x_i)$ is the convex function. This identity originates from the more general Bregman divergence[8], of which the KL-divergence is a special case.

Let $\mathbf{x}^*$ be the unique optimal solution (uniqueness guaranteed by closed and convex set $\mathcal{C}$). Applying the identity with $\mathbf{x} = \mathbf{x}^*$, $\mathbf{y} = \mathbf{x}^{(k+1)}$, $\mathbf{z} = \mathbf{x}^{(k)}$, and noting that $\mathbf{x}^* \in \mathcal{C} \subset \mathcal{C}_l$, we obtain:

$$\mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k)}) = \mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k+1)}) + \mathrm{KL}(\mathbf{x}^{(k+1)} \mid \mathbf{x}^{(k)}) + \langle \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}), \mathbf{x}^* - \mathbf{x}^{(k+1)} \rangle. \tag{21}$$

The iteration $\mathbf{x}^{(k+1)}$ satisfies the first-order optimality condition for Bregman projection onto the convex set,

$$\langle \nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}), \mathbf{x} - \mathbf{x}^{(k+1)} \rangle \geq 0, \ \forall \mathbf{x} \in \mathcal{C}_l. \tag{22}$$

Substituting $\mathbf{x} = \mathbf{x}^*$ into 22 implies that the inner product term in (21) is nonnegative. Thus,

$$\mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k)}) \geq \mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k+1)}) + \mathrm{KL}(\mathbf{x}^{(k+1)} \mid \mathbf{x}^{(k)}). \tag{23}$$

It follows that the sequence $\{\mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k)})\}$ is non-increasing and bounded below by zero, hence convergent.
Summing (23) over k yields:

$$\sum_{k=0}^{\infty} \mathrm{KL}(\mathbf{x}^{(k+1)} \mid \mathbf{x}^{(k)}) \leq \mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(0)}) < \infty, \tag{24}$$

which implies $\mathrm{KL}\left(\mathbf{x}^{(k+1)} \mid \mathbf{x}^{(k)}\right) \to 0$ as $k \to \infty$. By leveraging the convexity property of the KL-divergence, it can ensure $\left\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\right\| \to 0$.

Since the sequence $\left\{\mathbf{x}^{(k)}\right\}$ is bounded, by the Bolzano-Weierstrass theorem, there exists a convergent subsequence $\left\{\mathbf{x}^{(k_j)}\right\} \to \bar{\mathbf{x}}$. By the continuity of the projection operator, we have $\bar{\mathbf{x}} \in \mathcal{C}_l$ for all l, and it thus follows that $\bar{\mathbf{x}} \in \mathcal{C}$.

Because the sequence $\{\mathrm{KL}(\mathbf{x}^* \mid \mathbf{x}^{(k)})\}$ converges and $\mathrm{KL}\left(\bar{\mathbf{x}} \mid \mathbf{x}^{(k)}\right) \to 0$, it follows that $\mathbf{x}^{(k)}$ converges asymptotically to $\bar{\mathbf{x}}$. Therefore, $\bar{\mathbf{x}}$ is a solution to Eq. 7, i.e., $\bar{\mathbf{x}} = \mathbf{x}^*$. Moreover, since the optimal solution $\mathbf{x}^*$ is unique, all convergent subsequences must converge to the same limit. Therefore, the entire sequence $\left\{\mathbf{x}^{(k)}\right\}$ converges to $\mathbf{x}^*$. □

Proposition 2. *The sequence $\left\{\mathbf{x}^{(k)}\right\}$ generated by the Bregman iterative projection method converges superlinearly to the solution $\mathbf{x}^*$.*

Proof. Define the iteration function

$$f(\mathbf{x}) = (\mathbf{1}_n - \mathbf{w}_l) \odot \mathbf{x} + \mathbf{w}_l \odot \frac{\mathbf{b}_l}{\mathbf{w}_l^T \mathbf{x}} \mathbf{x}. \tag{25}$$

Assume that $\lim_{k \to +\infty} \mathbf{x}^{(k)} = \mathbf{x}^*$, then $f(\mathbf{x}^*) = \mathbf{x}^*$ and $\mathbf{w}_l^T \mathbf{x}^* = \mathbf{b}_l$.

Perform a Taylor expansion of $\mathbf{x}^{(k+1)} = f(\mathbf{x}^{(k)})$ around $\mathbf{x}^*$:

$$\mathbf{x}^{(k+1)} = f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)^T (\mathbf{x}^{(k)} - \mathbf{x}^*) + o(\left\| \mathbf{x}^{(k)} - \mathbf{x}^* \right\|_2),$$

where Jacobian matrix $\nabla f(\mathbf{x}^*)^T$ can be computed as follows:

$$\nabla f(\mathbf{x}^*)^T = \operatorname{diag}(\mathbf{1}_n - \mathbf{w}_l) + (\mathbf{w}_l \odot \mathbf{x}^*) \nabla^T \left(\frac{\mathbf{b}_l}{\mathbf{w}_l^T \mathbf{x}^*} \right) + \frac{\mathbf{b}_l}{\mathbf{w}_l^T \mathbf{x}^*} \operatorname{diag}(\mathbf{w}_l) \tag{26}$$

$$= \mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T. \tag{27}$$

Now, consider the ratio of successive errors in the L2 norm. Let $e^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$. From (25) and (26),

$$\frac{\left\| \mathbf{x}^{(k+1)} - \mathbf{x}^* \right\|_2}{\left\| \mathbf{x}^{(k)} - \mathbf{x}^* \right\|_2} = \frac{\left\| \nabla f(\mathbf{x}^*)^T (\mathbf{x}^{(k)} - \mathbf{x}^*) + o(\left\| e^{(k)} \right\|_2) \right\|_2}{\left\| e^{(k)} \right\|_2} \tag{28}$$

$$= \frac{\left\| (\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T) e^{(k)} + o(\left\| e^{(k)} \right\|_2) \right\|_2}{\left\| e^{(k)} \right\|_2} \tag{29}$$

$$= \frac{\left\| (\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T) e^{(k)} \right\|_2}{\left\| e^{(k)} \right\|_2}. \tag{30}$$

Thus, the squared ratio satisfies

$$\lim_{k \to +\infty} \frac{\left\| \mathbf{x}^{(k+1)} - \mathbf{x}^* \right\|_2^2}{\left\| \mathbf{x}^{(k)} - \mathbf{x}^* \right\|_2^2} = \lim_{e^{(k)} \to 0} \frac{((\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T) e^{(k)})^T ((\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T) e^{(k)})}{e^{(k)T} e^{(k)}} \tag{31}$$

$$= \lim_{e^{(k)} \to 0} \frac{e^{(k)T} (\mathbf{I}_n - \mathbf{w}_l \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l}) (\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T) e^{(k)}}{e^{(k)T} e^{(k)}}. \tag{32}$$

This limit depends only on the direction of $e^{(k)}$, not its magnitude. Define

$$M = (\mathbf{I}_n - \mathbf{w}_l \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l}) (\mathbf{I}_n - \frac{(\mathbf{w}_l \odot \mathbf{x}^*)}{\mathbf{b}_l} \mathbf{w}_l^T), \tag{33}$$

and Let λ be an eigenvalue of M with corresponding eigenvector $\mathbf{p}$. To find all eigenvalues and eigenvectors of M, set

$$\mathbf{u} := \mathbf{w}_l \in \{0, 1\}^n, \quad \mathbf{v} := \frac{\mathbf{w}_l \odot \mathbf{x}^*}{\mathbf{b}_l}.$$

Then $\mathbf{u}^T \mathbf{v} = \mathbf{v}^T \mathbf{u} = 1$.

Denote

$$s := \mathbf{u}^T \mathbf{u} = \|\mathbf{u}\|^2, \quad \alpha := \mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|^2.$$

If a vector $\mathbf{z}$ satisfies $\mathbf{u}^T \mathbf{z} = 0$ and $\mathbf{v}^T \mathbf{z} = 0$ (i.e., $\mathbf{z}$ is orthogonal to both $\mathbf{u}$ and $\mathbf{v}$), then

$$(\mathbf{I}_n - \mathbf{v}\mathbf{u}^T)\mathbf{z} = \mathbf{z}, \quad (\mathbf{I}_n - \mathbf{u}\mathbf{v}^T)\mathbf{z} = \mathbf{z},$$

hence

$$M\mathbf{z} = (\mathbf{I}_n - \mathbf{u}\mathbf{v}^T)(\mathbf{I}_n - \mathbf{v}\mathbf{u}^T)\mathbf{z} = \mathbf{z}.$$

Thus any vector in $\operatorname{span}(\mathbf{u}, \mathbf{v})$ is an eigenvector with eigenvalue 1.

Since

$$(\mathbf{I}_n - \mathbf{v}\mathbf{u}^T)\mathbf{v} = \mathbf{v} - (\mathbf{u}^T\mathbf{v})\mathbf{v} = \mathbf{v} - \mathbf{v} = 0,$$

we have

$$M\mathbf{v} = 0,$$

so $\mathbf{v}$ is an eigenvector with eigenvalue 0. Moreover,

$$(\mathbf{I}_n - \mathbf{v}\mathbf{u}^T)\mathbf{u} = \mathbf{u} - (\mathbf{u}^T\mathbf{u})\mathbf{v} = \mathbf{u} - s\mathbf{v}.$$

Then

$$\begin{aligned} M\mathbf{u} &= (\mathbf{I}_n - \mathbf{u}\mathbf{v}^T)(\mathbf{u} - s\mathbf{v}) \\ &= (\mathbf{u} - s\mathbf{v}) - \mathbf{u}\mathbf{v}^T(\mathbf{u} - s\mathbf{v}). \end{aligned}$$

Since $\mathbf{v}^T\mathbf{u} = 1$, we have $\mathbf{v}^T(\mathbf{u} - s\mathbf{v}) = 1 - s\alpha$. Therefore,

$$M\mathbf{u} = \mathbf{u} - s\mathbf{v} - \mathbf{u}(1 - s\alpha) = s\alpha\mathbf{u} - s\mathbf{v}.$$

In the basis $(\mathbf{u}, \mathbf{v})$, consider an eigenvector of the form $\mathbf{u} - k\mathbf{v}$. Then $M(\mathbf{u} - k\mathbf{v}) = \lambda(\mathbf{u} - k\mathbf{v})$ implies $M\mathbf{u} = \lambda(\mathbf{u} - k\mathbf{v})$. Solving this yields the eigenvector $\mathbf{u} - \frac{1}{\alpha}\mathbf{v}$ with eigenvalue $s\alpha$.

By properties of the Rayleigh quotient, the limit in (8) lies between the minimum and maximum eigenvalues of M, depending on the direction of $e^{(k)}$. The limit attains the minimum eigenvalue 0 if and only if $e^{(k)}$ is parallel to the eigenvector corresponding to 0, i.e., $\mathbf{v} = \frac{\mathbf{w}_l \odot \mathbf{x}^*}{\mathbf{b}_l}$.

From the iteration formula,

$$\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)} = (\mathbf{w}_l \odot \mathbf{x}^{(k)}) \left(\frac{\mathbf{b}_l}{\mathbf{w}_l^T \mathbf{x}^{(k)}} - 1 \right).$$

Thus $\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ is always parallel to $\mathbf{w}_l \odot \mathbf{x}^{(k)}$. Since $\lim_{k \to +\infty} \mathbf{x}^{(k)} = \mathbf{x}^*$, for sufficiently large k we have $\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}$ parallel to $\mathbf{w}_l \odot \mathbf{x}^*$, and hence $e^{(k)}$ becomes parallel to $\mathbf{v}$. Therefore, in the limit, $e^{(k)}$ is parallel to the eigenvector of M with eigenvalue 0, and consequently the limitation in (31) equals 0. That is

$$\lim_{k \to +\infty} \frac{\left\| \mathbf{x}^{(k+1)} - \mathbf{x}^* \right\|_2^2}{\left\| \mathbf{x}^{(k)} - \mathbf{x}^* \right\|_2^2} = 0,$$

which implies that the sequence $\{\mathbf{x}^{(k)}\}$ converges superlinearly in the L2 norm.

□

F Additional Theoretical Discussions

F.1 On the Binary Assumption in BLCLayer

The proposed LinConLayer framework consists of two complementary branches:

- **BLCLayer**, which enforces linear constraints with binary coefficients via KL-divergence based Bregman projections;
- **GLCLayer**, which enforces general linear equality constraints via a Euclidean projection with a closed-form solution.

The assumption that the constraint matrix W is binary is imposed *only* for BLCLayer and is made for technical and efficiency reasons. It is not a fundamental limitation of the overall framework: for non-binary or continuous linear constraints one can simply use GLCLayer, which handles arbitrary real-valued coefficients.

From an application standpoint, binary constraint matrices appear naturally in many structured prediction tasks, especially those with probabilistic outputs and "set selection / counting" structure, such as classification, (partial) matching, and graph-based problems. In these settings the constraints typically encode which entries must be selected or that row/column sums must equal prescribed cardinalities, so a 0–1 structure for W is a natural modeling choice rather than an artificial restriction.

The binary structure also enables particularly efficient Bregman projections. In BLCLayer each constraint row $w_\ell^\top x = b_\ell$ admits an elementwise update in closed form, so that one Bregman step reduces to simple elementwise multiplications and divisions in the spirit of Sinkhorn iterations. This is crucial for the strong empirical efficiency observed in our graph matching and portfolio experiments.

While BLCLayer is designed for the binary case, several directions suggest how one might extend the KL-based branch to more general coefficient patterns:

- **Row decomposition / reparameterization.** For some constraint rows w_ℓ, the coefficients can be expressed as a linear combination of a small number of weight groups (e.g., variables sharing the same pattern). Introducing auxiliary variables then allows one to decompose such a row into several 0–1 selection constraints, thereby retaining the fast BLCLayer updates while preserving the continuous information in the original coefficients.
- **One-dimensional convex subproblems under KL.** In the non-binary case, the Bregman projection associated with a single constraint row reduces to a one-dimensional convex optimization problem. This subproblem can be solved numerically, e.g., by a few Newton or bisection iterations, and different rows remain fully parallelizable. This yields an intermediate design point between binary BLCLayer and the fully closed-form GLCLayer, preserving KL geometry while supporting general W.
- **Hybrid designs.** Within the same network, one can use BLCLayer for naturally 0–1 structural constraints (e.g., combinatorial or cardinality constraints) and GLCLayer for general real-valued physical or economic constraints. This hybrid allocation strategy highlights the practical flexibility of LinConLayer when modeling mixed constraint structures.

A full development of these ideas is left for future work; here we include them to clarify the scope of the current design and to indicate promising extensions.

F.2 Extensions beyond Linear Constraints

The main paper focuses on linear constraints of the form

$$\mathcal{C} = \{x \in \mathbb{R}^n : Wx = b,\ x \geq 0\}.$$

Conceptually, the same implicit-layer philosophy could be applied to more general constraint sets, such as quadratic, nonlinear, or logical constraints, but this raises several design and numerical challenges.

As a simple example, consider a quadratic constraint

$$g(x) \leq 0, \quad g(x) = x^\top Q x + r^\top x + s.$$

Given an unconstrained network output y, one could in principle enforce $g(x) \leq 0$ by solving the projection problem

$$\min_x \ D(x, y) \quad \text{s.t.} \quad g(x) \leq 0,$$

where D is a divergence such as KL or a squared Euclidean distance. A straightforward implementation would introduce a penalty parameter $\lambda > 0$ and consider

$$\min_x \ D(x, y) + \lambda\, \phi(g(x)),$$

for some convex penalty ϕ, and solve the resulting unconstrained problem by gradient-based iterations.

However, this naive strategy has several drawbacks when used as a neural network layer:

- It introduces additional hyperparameters (penalties, step sizes, stopping criteria), making both training and tuning more cumbersome.
- It typically does not admit a closed-form solution, complicating convergence analysis and the stability of backpropagation.
- It can significantly increase computational cost compared to the simple closed-form or Bregman updates used in our linear setting.

The key challenge in extending LinConLayer beyond linear constraints is therefore to design implicit layers that (i) avoid unnecessary hyperparameters, (ii) admit well-behaved gradients, and (iii) remain computationally efficient. Exploring such generalizations—for instance via more general differentiable convex solvers plugged into the same implicit framework—is an interesting direction for future work, but lies outside the scope of the present paper.

F.3 Universal Approximation with Projection Layers

A natural question is whether appending a fixed projection layer at the network output reduces the representational power of the model. Here we provide a short argument showing that, for constraint-satisfying target mappings, the projection does not weaken universal approximation properties.

Let $\mathcal{C} = \{x \in \mathbb{R}^n : Wx = b,\ x \geq 0\}$ denote the feasible set defined by the linear constraints, and let $\Pi_{\mathcal{C}}$ be the Euclidean projection onto $\mathcal{C}$,

$$\Pi_{\mathcal{C}}(y) = \arg\min_{x \in \mathcal{C}} \|x - y\|_2^2.$$

It is well known that $\Pi_{\mathcal{C}}$ is 1-Lipschitz and acts as the identity on $\mathcal{C}$. Consider the class of continuous target mappings

$$\mathcal{F}_{\mathcal{C}} := \{f : \mathcal{X} \to \mathcal{C} \mid f \text{ is continuous}\}.$$

Let $\{g_\theta\}_\theta$ be a standard neural network family (e.g., a multilayer perceptron or a GNN) with the universal approximation property on $\mathcal{X}$: for any continuous $f : \mathcal{X} \to \mathbb{R}^n$ and any $\varepsilon > 0$, there exists θ such that

$$\sup_{x \in \mathcal{X}} \|g_\theta(x) - f(x)\|_2 \leq \varepsilon.$$

Lemma. *For any $f \in \mathcal{F}_{\mathcal{C}}$ and any $\varepsilon > 0$, there exists a parameter θ such that*

$$\sup_{x \in \mathcal{X}} \left\|\Pi_{\mathcal{C}}(g_\theta(x)) - f(x)\right\|_2 \leq \varepsilon.$$

Sketch of proof. Fix $f \in \mathcal{F}_{\mathcal{C}}$ and $\varepsilon > 0$. By universal approximation of $\{g_\theta\}$, there exists θ such that

$$\sup_{x \in \mathcal{X}} \|g_\theta(x) - f(x)\|_2 \leq \varepsilon.$$

Since $f(x) \in \mathcal{C}$ for all x, we have $\Pi_{\mathcal{C}}(f(x)) = f(x)$. Using the 1-Lipschitz property of Euclidean projection,

$$\|\Pi_{\mathcal{C}}(g_\theta(x)) - f(x)\|_2 = \|\Pi_{\mathcal{C}}(g_\theta(x)) - \Pi_{\mathcal{C}}(f(x))\|_2 \leq \|g_\theta(x) - f(x)\|_2 \leq \varepsilon,$$

for all $x \in \mathcal{X}$. Taking the supremum over $\mathcal{X}$ yields the claim.

This shows that, when the target mapping is required to satisfy the linear constraints by design (i.e., $f(\mathcal{X}) \subseteq \mathcal{C}$), composing a universal base network with the projection $\Pi_{\mathcal{C}}$ preserves universal approximation. The projection layer prevents infeasible outputs but does not exclude any constraint-satisfying target functions. This is consistent with universal approximation results for other projection-based architectures and places LinConLayer within the same theoretical landscape.